
WANDR: A Benchmark for Wide and Deep Research

Vitalii Polshkov* Marcin Pitera Jeremy Yang*

Kirill Priemko Maksim Gaiduk Aleks Nikolenko

Denis Bykov Denis Yarats Clare Southern Jerry Ma

Perplexity

July 14, 2026

Abstract

WANDR (Wide ANd Deep Research) is a benchmark of 500 realistic and challenging agentic data-collection tasks for knowledge work. Each task asks a system to discover a large collection of members satisfying specified criteria (the *wide* axis) and establish specific facts about each member through orchestrated web searches (the *deep* axis). We represent a task as a qualification key hierarchy such as `company (n) → employee (m) → url (k)`, meaning “find n qualifying companies, m qualifying employees per company, and k qualifying source pages per employee.” This hierarchy defines a target of $n \times m \times k$ records. Instead of comparing submissions against a static gold answer set, a task-specific judge re-fetches the cited pages and verifies each record’s claims against the cited evidence, allowing the benchmark to cover current and changing facts. Record-level verdicts are aggregated into soft and hard precision, recall, and F1 at the task level. Required record counts range from dozens to thousands, shifting the central challenge from finding a single correct answer or writing a polished report to open-ended discovery at high volume without sacrificing per-record factual correctness. Tasks are derived from de-identified product-usage logs and produced by a semi-automated pipeline. Automated checks and empirical audits screen them, with human review where needed; the final set is curated to stress-test recurring failure modes of frontier agentic search systems. The benchmark remains far from saturated: at a high effort setting, the strongest system achieves only 0.363 soft F1 and 0.133 hard F1. The tasks and evaluation harness are available at <https://github.com/ppl-ai/wandr>.

1 Introduction

Research agents are increasingly asked to produce not only a single answer or free-form report, but also a structured collection of facts, each requiring in-depth search to establish. This task shape recurs across the professional knowledge work observed in our production data, including competitive-landscape mapping, deal due diligence, literature review, market and product comparisons, and talent sourcing. It has two orthogonal dimensions: wide and deep. On the wide axis, the agent usually must discover a large set of entities not supplied in advance; some tasks instead specify a closed roster and require complete coverage. On the deep axis, it must enrich each entity through multiple orchestrated web searches. A representative request asks the agent to find at least 70 CEO or CFO appointments at US-based companies, each first announced during March–April 2026, and provide an authoritative appointment source for every company–appointee pair. A wide-and-deep research system returns such a structured collection, with one independently verifiable record per claim. WANDR tasks require dozens to thousands of such records. The central challenge is to achieve the requested volume through broad search while keeping every record factually correct.

To capture the variety of these use cases while keeping every claim independently verifiable, we represent each WANDR task as a flexible *qualification key hierarchy*: a tree of identifying keys and filtering criteria. The running example, `ceo_cfo_appointments`, has a primary hierarchy `company (70) → company_appointee (1) → url (1)`. More generally, each level may require several children— n companies, m appointees per company, and

*Correspondence to {vitaliy.polshkov, jeremy.yang}@perplexity.ai. We thank Omar Khattab for helpful suggestions.

k sources per appointee—so the hierarchy defines a target of $n \times m \times k$ records. Each record, the atomic unit of evaluation, cites a live page and includes verbatim excerpts, making it independently checkable.

At this scale, current agentic search systems often fail in predictable ways. We observe several recurring modes in production agent runs, some of which are also documented in prior literature: *volume collapse* (incomplete query decomposition and insufficient retrieval at scale, sometimes followed by hallucinated values) [Lan et al., 2025, Wong et al., 2025]; *snippet reliance* (using a search-result snippet without validating that its source supports the submitted claim) [Wong et al., 2025]; *non-systematic constraint application* (applying a filter to the first few entities but silently dropping it thereafter); *cross-reference skips* (asserting a claim without corroborating it across the required sources); *premature stopping* (giving up after an initial search yields insufficient information) [Wong et al., 2025]; *missing enrichment* (finding an entity but never gathering its required facts); and *context overflow* (long search trajectories exhausting the available context) [Lan et al., 2025]. WANDR originated as an internal benchmark designed to distinguish systems by how well they avoid these failure modes. In particular, it compares a *Search as Code* (SaC) system, in which agents compose programmable retrieval primitives [Perplexity, 2026], with conventional systems that distribute fixed-endpoint search-and-read loops across parallel subagents. SaC, the strongest system evaluated, achieves only 0.363 soft F1 and 0.133 hard F1, making the benchmark challenging even for frontier research agents.

The released task set instantiates recurring patterns from production data as self-contained benchmark tasks. Every admitted task must meet three top-level criteria: *substantial volume* (usually hundreds of records and sometimes thousands); *feasibility* (the requested volume is attainable); and *difficulty and discrimination* (simple internal baselines score low while stronger systems score meaningfully higher). Together, these choices yield five defining characteristics whose combination distinguishes WANDR:

- **Task shapes and topics grounded in production usage.** The released tasks are derived from de-identified requests observed in a real research product, preserving the domains, constraints, and wide-then-deep structures of professional knowledge work (Section 4).
- **A compositional tree structure that flexibly encodes task shapes and makes failures localizable.** A flat list and a fixed-schema table are both special cases of the qualification key hierarchy. Failures can be localized across the tree: a system may fail to discover core task entities or members, enrich intermediate entities or keys, identify relevant pages, extract adequate evidence from those pages, or disambiguate identities when keys collide (Section 3).
- **A semi-automated task-construction pipeline.** Automated authoring checks, an empirical feasibility audit, and a judge audit establish the conditions needed for a functional task. Optional human review improves task quality but is not required for the task package to run and grade submissions. With no gold answer set to annotate, human effort is quality review rather than exhaustive answer construction (Section 4).
- **Reference-free, evidence-verified grading.** Every record is a citation-backed claim (URL plus verbatim excerpts). A task-specific judge re-fetches the cited page. The full verdict evaluates both the page and submitted excerpts, while the retrieval-only verdict asks only whether the page satisfies every substantive task requirement. These verdicts are then aggregated into precision (accuracy among submitted members), recall (quality-adjusted completion relative to the required member count), and F1, each in soft (partial credit) and hard (binary pass/fail) forms (Section 5, Appendix A).
- **A scalable substrate for reinforcement learning (RL).** The task pipeline can generate diverse training packages; per-record verdicts and hierarchical metrics provide dense partial-progress rewards; per-level required counts provide an explicit difficulty gradient for curriculum learning; and the streaming, cache-aware grader can amortize overlapping work across training batches (Appendix I).

The rest of the paper is organized as follows. Section 2 situates WANDR among single-answer, collection, and domain-specialized benchmark families and compares how they establish ground truth. Section 3 defines the task structure and its variations; Section 4 describes the task-construction pipeline and reports summary statistics for the released set; Section 5 details how submissions are graded; Section 6 evaluates six production systems; and Section 7 summarizes the findings and discusses their implications, limitations, and future directions.

2 Related Work

We organize prior work on agentic information-seeking benchmarks primarily by output shape—a compact answer or a collection of items—and by how that output is graded. Domain specialization is a cross-cutting distinction: both compact-answer and collection benchmarks may restrict their tasks to one expert field.

Closed-ended browsing and question answering The first family asks an agent to solve a bounded browsing or question-answering problem and return a compact response. BrowseComp poses deliberately obscure single-answer questions [Wei et al., 2025]; Humanity’s Last Exam targets expert-level, closed-ended academic questions [Phan et al., 2025]; FRAMES and GAIA combine retrieval with multi-hop reasoning [Krishna et al., 2025, Mialon et al., 2024]; AssistantBench includes realistic web-research requests but permits at most five answers [Yoran et al., 2024]; and VeriWeb decomposes long-horizon web tasks into chains of individually verifiable subtasks [Liu et al., 2025]. These benchmarks stress depth or end-to-end task completion, but their compact, gold-answer-based outputs do not test repeated depth across a large collection of discovered entities. WANDR instead makes that width–depth composition the scored object: every member must be discovered, enriched, and independently evidenced.

Wide and broad information seeking Closest to WANDR are benchmarks whose deliverable is a collection of items graded individually. WideSearch [Wong et al., 2025] also sources tasks from real user queries and grades populated tables cell by cell, but primarily tests broad collection with comparatively shallow per-item fields. DeepSearchQA [Gupta et al., 2026] includes both single- and set-valued answers, graded against gold answer sets. WideSeekBench [Huang et al., 2026] uses a multi-stage generation and quality-control pipeline to produce table-completion tasks with varied target volumes and logical constraints. GISA [Zhu et al., 2026] spans item-, set-, list-, and table-valued answers and maintains a live subset through periodically updated answers. These benchmarks expand the width axis, but generally do not require the same multi-stage enrichment and corroboration process to be repeated for every discovered member. Most similar to WANDR, DeepWideSearch [Lan et al., 2025] explicitly combines depth and width, converting 220 tasks from existing deep- and wide-search datasets and grading against human-verified ground-truth tables. WANDR differs in task shape and grading: its requests require wide-then-deep search to be repeated within one hierarchy, rather than combining separately sourced deep and wide task patterns; its grading verifies claims against submitted sources rather than relying on a gold solution. The related *Table-as-Search* framework [Lan et al., 2026] casts wide-and-deep search as table completion, though as a solving method rather than a benchmark.

Domain-specialized information seeking Orthogonal to output format, a parallel line of work restricts search and research benchmarks to a single expert domain, testing domain knowledge alongside retrieval: medicine [Chen et al., 2025], finance [Zhu et al., 2025, Bigeard et al., 2025], law [Li et al., 2025], and academic literature [Zhou et al., 2025]. AutoResearchBench is also literature-specific: its *Deep Research* track identifies a target paper, while its *Wide Research* track collects all papers satisfying given conditions [Xiong et al., 2026]. These benchmarks raise the expertise bar and often constrain the sources or tools used within one field. WANDR is instead domain-general: it applies a common hierarchical representation and grading framework to varied task structures across talent, finance, health, legal and regulatory work, and other professional domains.

Grading paradigms Most prior benchmarks establish ground truth in one of three ways: matching gold labels for single-answer question answering; applying task-specific rubric criteria with a large language model (LLM) judge to long-form reports, as in our deep-research benchmark DRACO [Zhong et al., 2026]; or matching a gold collection in wide-search benchmarks. WANDR is the breadth-oriented counterpart to DRACO. Other work makes the evaluator itself agentic: Agent-as-a-Judge [Zhuge et al., 2024] evaluates code-agent outputs and trajectories against hierarchical requirements, while Mind2Web 2 [Gou et al., 2025] assesses answer correctness and source attribution for citation-backed responses. WANDR brings source verification to collection scale: every record must carry its own citation (URL plus verbatim excerpts), and a task-specific judge re-fetches the cited page and verifies the claim against it. Because the grader checks submitted claims rather than enumerating every eligible answer, this approach avoids exhaustive gold-answer annotation. Required-volume targets, validated for feasibility during task construction, provide the denominator for recall without requiring an enumerated answer set. This makes tasks about current and changing facts admissible and allows task production to scale through a semi-automated pipeline rather than exhaustive expert annotation. DeepWideSearch’s own limitations call for exactly this combination of “automated data generation techniques” and “reference-free evaluation metrics” [Lan et al., 2025].

Table 1 summarizes WANDR’s position. Relative to existing collection benchmarks, WANDR adds evidence-backed records, reference-free verification, and a compositional task structure: qualification key hierarchies in which discovery and per-entity enrichment form flexible trees rather than a single flat table. Its task shapes originate in real professional usage, and tasks are admitted only if they are high-volume, feasible, and discriminative against

internal baselines. The resulting challenge is to sustain both breadth and per-record accuracy across a large, structured collection—precisely the regime in which current agents struggle.

Benchmark	Real-world tasks	Open-set discovery	Collection-record grading	At-scale quality task generation	Reference-free verification
<i>Closed-ended browsing & question answering (task-level outputs)</i>					
GAIA [Mialon et al., 2024]	✗	✗	✗	✗	✗
BrowseComp [Wei et al., 2025]	✗	✗	✗	✗	✗
FRAMES [Krishna et al., 2025]	✗	✗	✗	✗	✗
AssistantBench [Yoran et al., 2024]	✗	✗	✗	✗	✗
VeriWeb [Liu et al., 2025]	✗	✗	✗	✗	✗
<i>Wide and broad information seeking (collections)</i>					
WideSearch [Wong et al., 2025]	✓	✓	✓	✗	✗
DeepWideSearch [Lan et al., 2025]	✗	✓	✓	✗	✗
DeepSearchQA [Gupta et al., 2026]	✗	✓	✓	✗	✗
AutoResearchBench (Wide) [Xiong et al., 2026]	✗	✓	✓	✗	✗
WideSeekBench [Huang et al., 2026]	✗	✓	✓	✓	✗
GISA [Zhu et al., 2026]	✗	✓	✓	✗	✗
WANDR (ours)	✓	✓	✓	✓	✓

Table 1: WANDR versus representative search and research benchmarks. Rows are grouped by output format; domain specialization is orthogonal, so AutoResearchBench’s Wide track appears with collection benchmarks. **Real-world tasks:** task shapes derived from real user requests or professional workflows rather than hand-curated puzzles or conversions of other benchmarks. **Open-set discovery:** the entities to report are not given and must be found at high recall. **Collection-record grading:** collection items, rows, or cells are a primary scoring unit, rather than a compact task-level answer or holistic report. **At-scale quality task generation:** a scalable generation process couples task creation with automated quality controls rather than relying only on manual task-by-task authoring. **Reference-free verification:** grading checks each claim against its fetched source, with no pre-annotated gold answer set to match. For mixed-format benchmarks, a check indicates that at least one named track or task family has the property.

3 Task Structure

3.1 Recurring Patterns in Production Requests

Wide-and-deep research requests vary widely in subject matter, but in the production data we observe a small number of recurring shapes. In *entity discovery*, the system must find many instances of an entity class, such as companies, products, people, or events. In *entity enrichment*, the entities are already known and the system must fill in the same facts for each one. The most common shape combines the two: first discover the entities, then investigate each one in depth. A fourth pattern is *multi-condition research*, in which every candidate must satisfy several conditions, often supported by different pages or source types.

These patterns also appear inside one another. A request may ask for companies, several products per company, several facts per product, and one or more sources for every fact. Other requests repeat a fixed set of facets for each entity, compare the same entities across time, or ask both sides of a relationship to document the connection. The surface form changes, but the intent structure is the same: for each qualifying item, find a required set of qualifying children and carry the process through to source-backed evidence.

3.2 A Common Tree Representation

WANDR represents this repeated structure flexibly as a tree. Each level names the kind of item being collected, and each edge means *for each parent, find these children*. Every branch ends in one or more URLs, so each claim remains tied to the page that supports it. This representation handles a wide range of real-world requests without defining a new output format for every workflow. The tree’s topology specifies which items to find, how they are related, and how many are required. Concretely, it defines the keys, the parent–child edges, the required count at each level, and any subtasks whose top key reuses a key in the primary hierarchy.

3.3 Canonical Form

A basic WANDR hierarchy has the form

$$\text{key}_1(r_1) \rightarrow \text{key}_2(r_2) \rightarrow \dots \rightarrow \text{key}_d(r_d) \rightarrow \text{url}(r_u).$$

A *key* is the field, or group of fields, that identifies an item at one level. The value in parentheses is the minimum number of distinct children required at that level. A count is therefore a coverage floor; a closed-set level instead supplies the complete allowed roster. The hierarchy always ends in `url`. The count is applied separately beneath each parent: the running example asks for one appointment under each of 70 companies, not 70 appointments distributed across fewer companies. A key may be composite when one field is not globally unambiguous; `company_appointee{company, appointee}` distinguishes appointments of the same person—or same-named people—across different companies.

A complete root-to-leaf path defines one record slot `{item, url, excerpts, answer}`: the item fields specify the key, the URL supplies its source page, excerpts are verbatim passages selected from the cited page, and answer is flexible JSON content describing the claim. A record is the atomic unit of grading. For corpus-level scale statistics, a *member* is the task’s designated core unit of coverage; in the running example, each company is one member and requires two leaf records, one in each branch. Counts apply recursively. A parent reaches its structural target when it has the required number of distinct children, and each child must in turn contain its own required descendants.

3.4 Running Example

We use the same `ceo_cfo_appointments` task for illustration.

ceo_cfo_appointments

Primary task: For at least 70 US-based companies, identify at least one CEO or CFO appointment first announced between March 1 and April 30, 2026, and provide an authoritative appointment page.

Primary record: one company–appointee appointment backed by an authoritative announcement page.

Primary validity and requirements: the company is US-based; the page identifies the company and appointee, establishes the CEO or CFO role and first public announcement within the target window, and comes from an authoritative issuer, filing, newswire, or directly attributed first-hand business-journalism surface.

Company-listing subtask: For the same 70 or more companies, supply at least one URL on a recognized listing-authority surface showing that the company has a primary or secondary listing on a US national securities exchange (NYSE, NASDAQ, NYSE American, or NYSE Arca) or is a US-domiciled SEC Exchange Act reporting issuer.

Subtask record: the same company backed by a listing-authority page.

Listing requirements:

1. The page identifies the claimed company.
2. The page communicates recognized listing-authority authorship, including through its URL when applicable.
3. The page shows the required US exchange listing or US-domiciled Exchange Act reporting status.

Records required: 140 (70 appointment records plus 70 listing records).

The primary hierarchy is

$$\text{company}(70) \rightarrow \text{company_appointee}\{\text{company}, \text{appointee}\}(1) \rightarrow \text{url}(1).$$

A subtask, `company_listings`, reuses the company key and adds one listing-authority URL per company. Its top key matches the primary root, so the two branches align at the company level and together require 70 appointment paths plus 70 listing paths.

This one task demonstrates open-set discovery, a composite key, recursive per-parent counts, and a subtask. The next subsection uses shorter examples to explain other structural variations.

3.5 Structural Variations

The canonical hierarchy is intentionally small, but changing what a level represents, how many children it requires, or where a new branch attaches covers a broad range of research workflows. Figure 2 highlights six common patterns; the four groups below define these patterns and related variations.

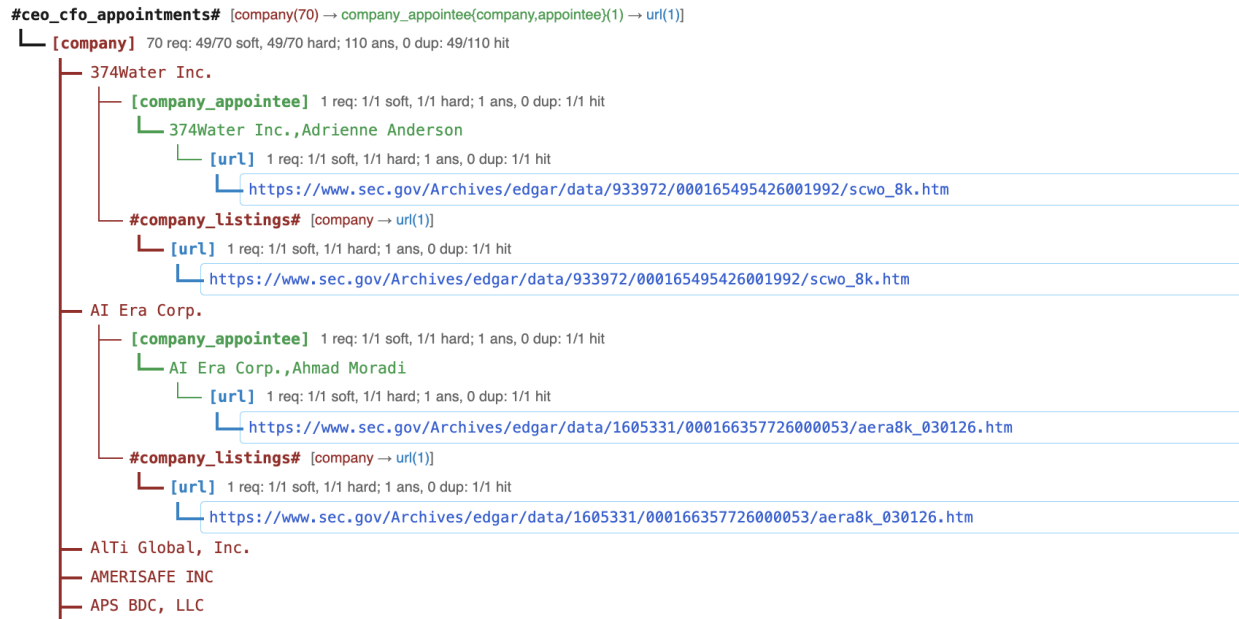


Figure 1: Task viewer for a graded `ceo_cfo_appointments` submission. The primary hierarchy requires 70 company–appointee records, and the `company_listings` branch attaches one listing-authority URL to each shared company. The expanded rows show both branches for 374Water and AI Era; the remaining companies are collapsed.

Open versus closed entity sets Most levels ask the system to discover a requested number of qualifying values. A closed-set level instead supplies the complete roster, changing “find n ” into “find all.” The hierarchy and scoring rules remain the same, but canonicalization and deduplication map submitted names to the supplied entities and enforce complete coverage; Appendix B details these mechanisms.

Organizing entities

- *Flat list*: one entity level sits above the URL leaves, as in `company( $n$ ) -> url(1)`.
- *Nesting*: levels follow a containment relationship, as in countries, cities within each country, and evidence for each city.
- *Fan-out*: a parent connects to several related entities that are not contained within it. The `misquotation_instances` task asks for two attributed authors and five pages per quote–author pair; a startup may fan out to several investors.
- *Matrix*: the same kind of child is repeated across many parents, often with an encouraged or fixed shared set. Examples include the same set of benchmarks for every model evaluated or the same set of products carried by every retailer.

Controlling coverage

- *Dispatch*: a level contains a fixed set of labels rather than discovered entities. The labels can represent facets, such as product, market, and customer review for a company. Requiring every label makes coverage explicit.
- *Anchor*: when later levels fan out, an intermediate key with required count one makes each selected entity the local reference point for its descendants. In `company( $n$ ) -> product(1) -> distributor(2)`, each company must have at least one product with two distinct distributors, both tied to that product. A distributor may recur under another product, but that relationship must be supported on its own path.
- *Partition*: a small top level divides the search space before the main entities are collected. `country(3) -> company( $m$ )` enforces geographic spread that a flat company list would not.

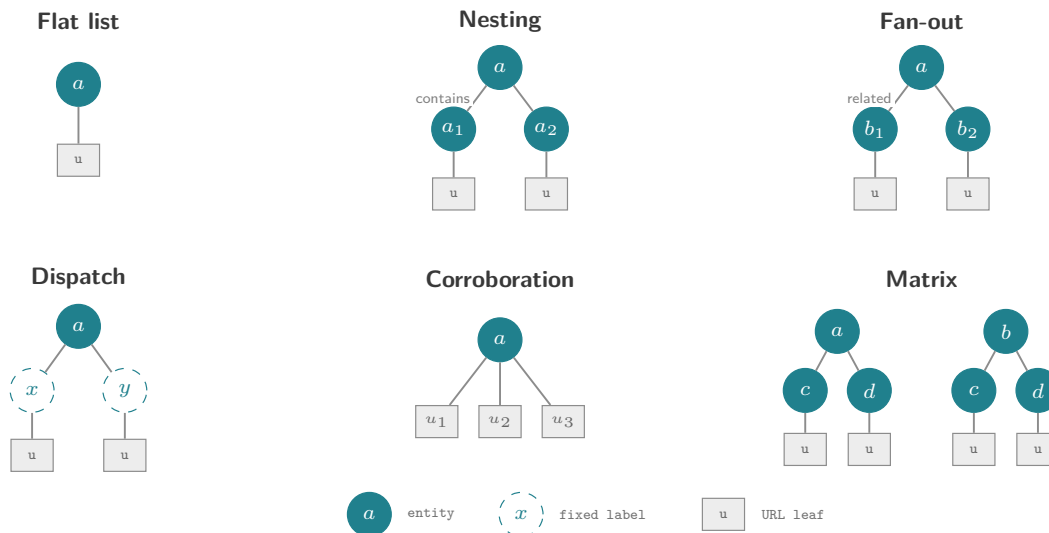


Figure 2: Six common hierarchy patterns. Filled circles are entities, dashed circles are fixed labels, and squares are URL leaves. In the matrix, repeated child labels denote the same child values under different parents.

Structuring evidence

- *Corroboration*: a URL count above one asks several distinct pages to support the same claim independently; each page must satisfy the full claim. A task may separately require source or domain independence.
- *Triangulation*: several pages jointly describe a quantity or market surface without having to report the same value, such as three current retailer prices.
- *Data as keys*: an extracted value becomes its own level when values must be counted, kept distinct, or held fixed across later evidence. A `price(1) → url(3)` branch requires all three pages to concern the same submitted price; a `signal(k)` level requires k distinct signals.

Adding subtasks A subtask reuses a parent key but adds another hierarchy. It usually serves one of four roles:

- *Enrichment*: add facts that are useful but separate from the primary claim, such as pricing for each model or a release paper for each benchmark.
- *Another angle*: investigate a related question about the same entity, such as open roles alongside evidence of a recent company strategy change.
- *Another source family*: require evidence from a distinct source universe, as the running CEO/CFO task does by separating appointment evidence from listing-authority evidence.
- *Chain*: attach a later branch to entities identified by an earlier one, such as `approval → active molecule → credited scientist → dissertation`.

These variations are building blocks, not mutually exclusive task classes. A single hierarchy can combine nesting, fan-out, fixed dispatch labels, and multiple-source requirements, while subtasks add further branches at any shared key. This composability allows one representation to cover diverse workflows.

4 Task Construction

Tasks are produced by a semi-automated pipeline with four stages: *seeding*, iterative *authoring*, *admission* into a candidate pool, and *curation* into the released set (Figure 3).

Seeding identifies requests with a wide-research shape in de-identified product-usage logs. During *authoring*, an author agent drafts the design, task specification, and fixtures, and a separate *critic* reviews every draft; failures return

to the stage that owns them. *Admission* checks that the requested volume is attainable, audits the judge, and optionally adds human sign-off. Finally, *curation* labels admitted tasks and selects a subset matching the target distributions. Each released task ships as a self-contained *package* with its solver-facing task, verifier, fixtures, and labels. During construction, outputs from the authoring rollouts are merged into a best-known solution that serves as an internal feasibility witness, not an answer key; grading never compares a submission against it. Appendix C summarizes the public package format, while Appendix D expands the admission gates. The subsections below describe seeding (Section 4.1), authoring (Section 4.2), admission (Section 4.3), and curation (Section 4.4); Section 4.5 then summarizes the released set.

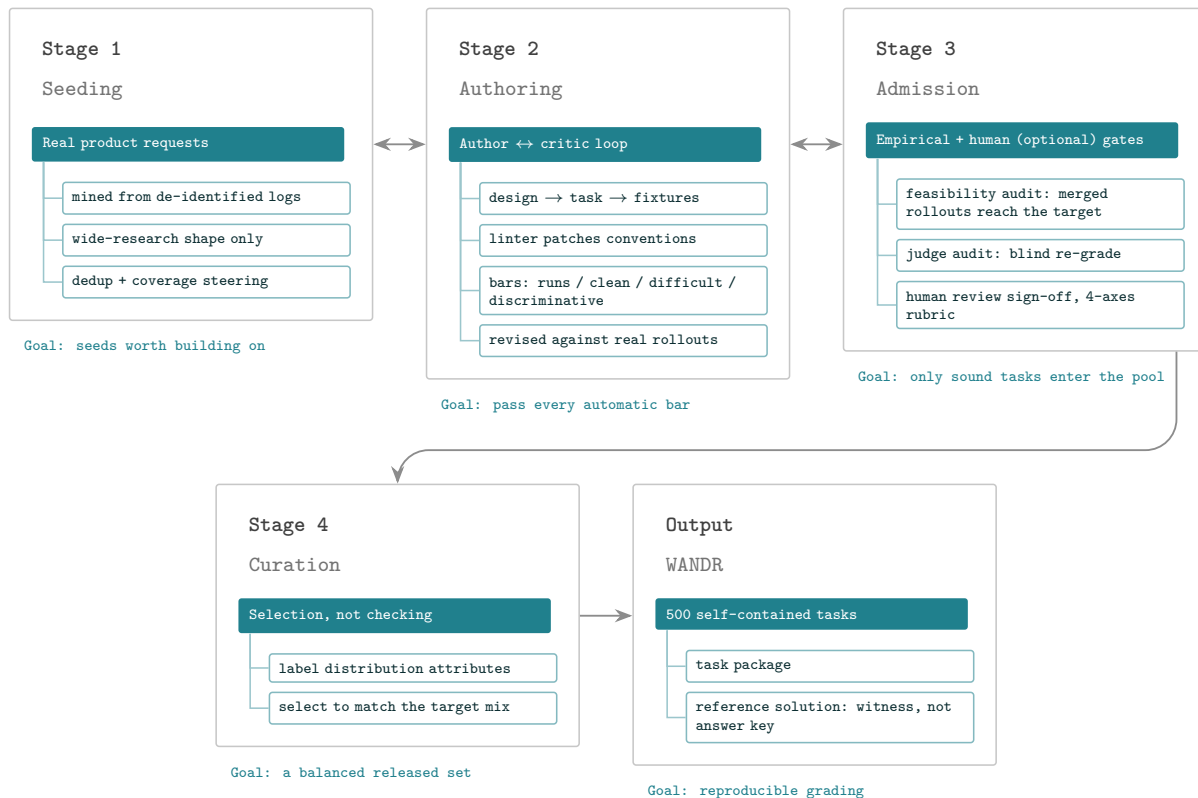


Figure 3: The task-construction pipeline has four stages. **Seeding** mines product logs for wide-research-shaped requests and selects a seed. **Authoring** uses an interleaved author–critic loop for design, task, and fixture drafts, plus a mechanical linter. **Admission** applies a feasibility audit based on merged rollouts and a judge audit, followed by optional human sign-off against the four-axis rubric; automated runnability, cleanliness, difficulty, and discrimination gates are enforced during authoring. **Curation** labels admitted tasks and selects a subset matching the target distributions. The output is 500 public task packages; system grading is a separate process (Section 5).

4.1 Seeding

Each task begins with a *seed*: a real request mined from de-identified product-usage logs.¹ A first filter retains requests shaped like wide research—asking to compile a list, table, or directory—and favors sessions with heavy tool use. A classifier then ranks candidates by whether their entities and sources are stable enough to build on, their answers can be checked, and discovery is genuinely difficult. Rejected seeds remain as negative examples so they do not resurface.

¹Two privacy safeguards apply. First, seed mining follows the de-identification methodology of the DRACO benchmark [Zhong et al., 2026]: requests are sampled from a de-identified pool and reworded to remove identifying details. Second, authoring retains only the seed’s reusable pattern and re-grounds the task in entities with a public web presence. A released task therefore reflects a recurring form of professional work, not the content of any individual query or private record.

The authoring stage claims seeds in batches under two controls: *deduplication* against previously used seeds and tasks, and *coverage steering* toward the target distributions, which favors diverse, economically useful domains and caps overrepresented or low-value ones.

4.2 Authoring

Authoring turns a seed’s reusable pattern into a fresh, self-contained task through an iterative loop between two agents: an *author* drafts, a separate *critic* reviews, and the cycle repeats until the draft passes the relevant checks or returns to an earlier stage. A structural problem can send the work back to seeding. When a draft is too easy or exposes a shortcut, the author uses the topology patterns in Section 3.5 and the criteria and identity mechanisms in Appendices A and B to raise its difficulty and adversarial robustness.² The work passes through three steps in order:³

- *Design*: the author sketches a few candidate directions for the task, with substantial web search to establish that the needed entities and pages actually exist; the critic selects one and sharpens it.
- *Task*: the author writes the task package—the key hierarchy, the description the solver reads, the judge specification, and the canonicalization/deduplication settings.
- *Fixture*: the author writes a few test records with known intended verdicts, grounded in real pages, and runs them through the judge to confirm the grader behaves as designed.⁴

Throughout the loop, four automated bars require that the package runs, its solver-facing description, judge specification, and schema remain aligned, primitive internal baselines do not already solve it, and stronger internal rollouts meaningfully outperform weaker ones. These are the runnability, cleanliness, difficulty, and discrimination checks shown in Figure 3.

The authoring context also contains a set of vetted guidance tasks. Appendix E profiles a subset of five, chosen to expose the rationale behind matrix, open-set, temporal-panel, legal-comparison, and reciprocal-evidence structures.

The pipeline can also create *sibling variants* by holding most of the hierarchy fixed while changing one controlled choice: a volume, time window, eligibility rule, or evidence bar. For example, `audio_gear` and its `audio_gear_relaxed` sibling share the same counts—50 products, two sentiment branches, and three URLs per branch—while the relaxed sibling accepts shorter and less dedicated opinion evidence. Other sibling pairs contrast current with historical scope. These pairs support sensitivity analysis without introducing a different task representation.

4.3 Admission

Admission begins after a task passes the automated runnability, cleanliness, difficulty, and discrimination gates during authoring. It adds a feasibility audit, a judge audit, and optional human review. A failed check returns the task to the stage that owns the problem.

Feasibility audit The pipeline merges everything found across 10–12 authoring rollouts into a single best-known solution, which must recover a near-full requested volume. A large surplus indicates that the task asks for too little; a persistent shortfall indicates that it asks for too much. This internal merged output is a feasibility witness, not an answer key. It establishes that the requested volume is attainable, while grading still evaluates every submission against its own cited pages rather than against the witness.

Judge audit A reviewer samples graded records and regrades each one blindly, forming an independent verdict before seeing the automated judgment. Disagreements are attributed to solver behavior, task design, or grading machinery. Any major defect blocks the task. After cases attributable to solver behavior are separated, an automated-judge error rate above approximately 10% of the remaining sample also blocks it.

²For example, if a seed permits a solver to cite mirror pages—duplicate copies of one listing on different domains—for every submission, the task is tightened to require distinct source domains per entity and prohibit template-substituted prose.

³The mechanical linter runs between steps when only mechanical work remains. It patches convention violations without changing the entity class, evidence bar, volume, or judge policy. A meaning-changing fix is returned to the Task or Design step instead.

⁴Fixtures are unit tests for the task’s judge; they ship with the task but never grade the systems under test. When a fixture the author trusts disagrees with the judge, the disagreement is triaged—usually the fixture is wrong and gets fixed, but a genuine judge defect sends the task back for repair.

Optional human review The author–critic loop automates much of the review; the human rubric provides a final checklist when sign-off is used. A reviewer scores four pass/fail axes: whether the *key hierarchy* has sensible keys, an attainable volume floor, and appropriate URL corroboration; whether the *task description* is complete and unambiguous; whether the *grader* correctly encodes the judge instructions, schema, validity gates, and requirement checks; and whether *identity handling* has appropriate canonicalization and deduplication settings. A reviewed task must pass all four axes. Appendix D gives the complete gate and review tables.

4.4 Curation

Curation selects a balanced release from the admitted pool. Because the pipeline deliberately overproduces, passing every check does not guarantee inclusion. Each admitted task receives distribution labels, and selection matches the target distributions; a strong task may remain unused when its bucket is already full. Selected tasks are then stamped into self-contained release packages. Across admission and curation, optional human work is budgeted at about 20 minutes per released task—approximately 15 minutes for rubric sign-off and 5 minutes for labeling—plus a roughly five-minute feasibility review for tasks that cannot be settled from merged rollouts. This is roughly one sixth of WideSearch’s reported 2.33-hour average for one human completion; that study additionally used two independent annotators per task [Wong et al., 2025]. Omitting optional human review allows task production to scale further.

4.5 Task Summary Statistics

We summarize the released benchmark by its subject areas, breadth, depth, hierarchy, and difficulty.

Verticals The release spans 13 vertical labels (Table 2). Each task has one to four labels: 294 tasks (58.8%) have one label, and 206 (41.2%) span multiple verticals. The mean is 1.50 labels per task.

Vertical	Tasks	Share
General	254	50.8%
Legal	168	33.6%
E-commerce	99	19.8%
Technology	66	13.2%
Events	55	11.0%
Finance	37	7.4%
Health	19	3.8%
Wikis	16	3.2%
Academic	15	3.0%
People	8	1.6%
Social	7	1.4%
Community	5	1.0%
Patents	3	0.6%

Table 2: Multi-label vertical coverage over all 500 benchmark tasks. Counts and shares are per label and therefore sum to more than 500 tasks and 100%.

Breadth vs. depth For these corpus-level statistics, a member is the task’s designated core unit of coverage, such as a company in the running example, whereas a record is one graded root-to-url path. Breadth is the number of required members per task, and depth is the number of required records per member; their product is the required record volume (Figure 4). The medians are 50 members, 4.00 records per member, and 245 records per task, respectively. Required volume is below 100 records for 10.8% of tasks, 100–299 for 44.4%, 300–999 for 40.4%, and at least 1,000 for 4.4%; across all tasks, the total is 170,495 records. Hierarchies are flat for 13.8% of tasks; 47.4% have one intermediate key, 32.2% have two, and 6.6% have three or more.

Difficulty Difficulty is calibrated from each task’s historical design-stage rollouts with an internal endpoint. For rollout r , let *soft-F1-avg* denote the arithmetic mean of soft F1 under the retrieval-only and full verdicts (Section 5, Appendix A); task difficulty is $1 - \max_r(\text{soft-F1-avg}_r)$. Sorting the 500 tasks by this value and splitting them by rank

Task scale and structure (N = 500 benchmark packages)

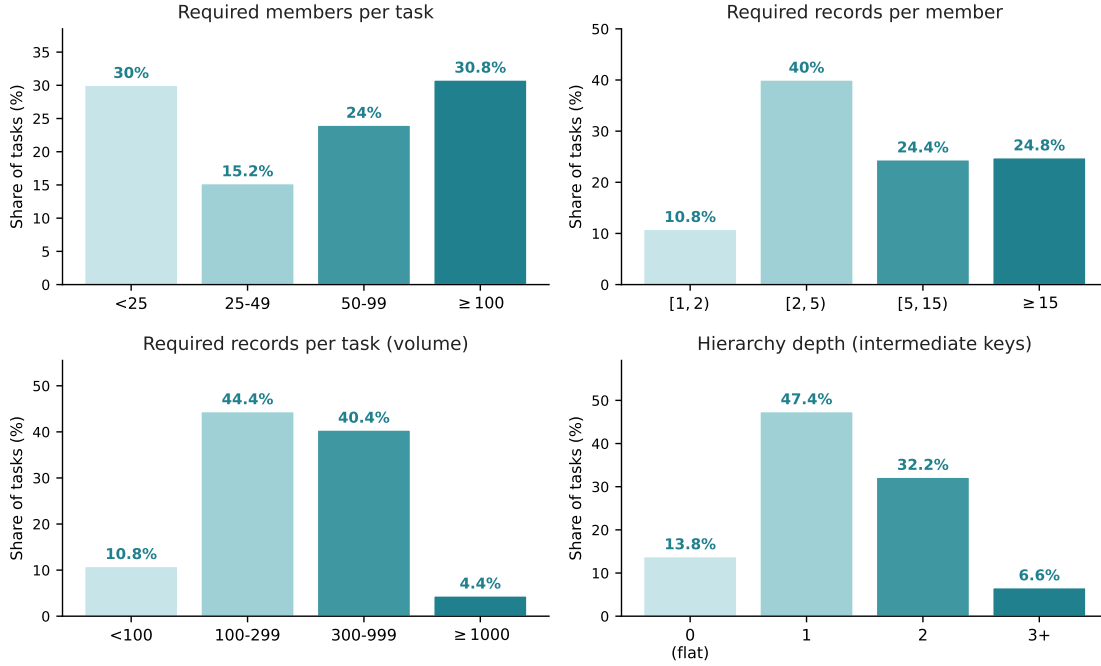


Figure 4: Required scale and structure of the 500-task benchmark set. Top left: required members from executed task configurations. Top right: required records per member. Bottom left: required volume; for each task this is exactly the product of the two top-panel quantities. Bottom right: intermediate keys between the member key and the `url` leaf.

yields 167 lower-, 166 middle-, and 167 higher-difficulty tasks. Figure 5 plots these relative tiers across the breadth–depth plane. Breadth and depth do not cleanly separate difficulty tiers because per-record work varies substantially.

5 Grading

Every leaf record receives an independent binary verdict, and the verdicts are then aggregated to the task level. The grader fetches the cited page and evaluates the submission `{item, url, excerpts, answer}` against universal and task-specific checks; a record passes only when every applicable condition holds. The judge prompt emits the repository field `verdict`; throughout the paper, we denote this complete-record verdict as `verdict_full`:

$$\text{verdict_full} = \underbrace{\mathbb{1}_{\text{page_content_usable}} \times \mathbb{1}_{\text{answer_intent_clear}} \times \mathbb{1}_{\text{excerpts_faithful}}}_{\text{universal}} \times \underbrace{\mathbb{1}_{\text{overall_valid}} \times \mathbb{1}_{\text{requirements_all_satisfied}} \times \mathbb{1}_{\text{requirements_all_supported}}}_{\text{task-specific}} \quad (1)$$

The universal block applies the same evidence mechanics to every task:

- `page_content_usable` requires substantive, on-topic page content.
- `answer_intent_clear` requires a specific, identifiable submitted claim.
- `excerpts_faithful` requires verbatim or near-verbatim, meaning-preserving excerpts.

The task-specific block combines eligibility and evidence:

- `overall_valid` requires a well-formed, in-scope record.
- `requirements_all_satisfied` asks whether the full page satisfies every substantive requirement.

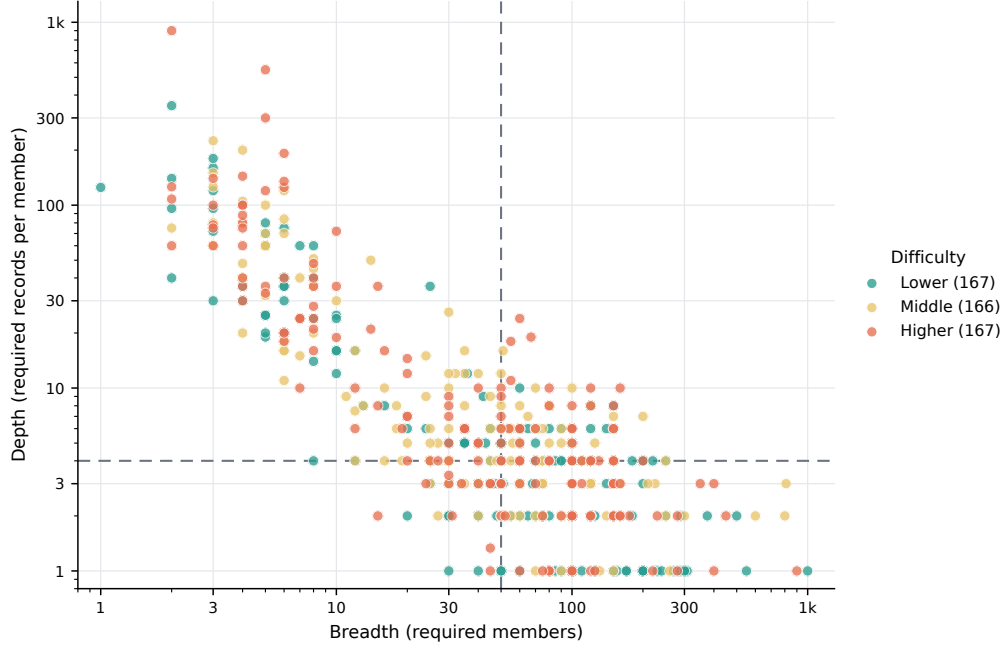


Figure 5: Overall task difficulty across the breadth–depth plane (log–log; 500 benchmark tasks). Breadth is the number of required members; depth is the required record count divided by that member count. Point color denotes the overall difficulty label. Dashed guides mark the median breadth (50 members) and median depth (4.00 records per member).

- `requirements_all_supported` asks whether the submitted excerpts alone support every requirement.

For comparison, the scorer defines a retrieval-only verdict:

$$\text{verdict_retrieval} = \mathbb{1}_{\text{requirements_all_satisfied}}. \quad (2)$$

The full verdict, `verdict_full`, evaluates the complete submitted record using the full page and submitted excerpts. The retrieval-only verdict, `verdict_retrieval`, asks only whether the fetched page satisfies every substantive task requirement, independently of the submitted excerpts. Universal and validity fields remain available as diagnostics but are not multiplied into `verdict_retrieval`. We use `verdict_full` for the main results and compare the two verdicts in Section 6.3. Appendix A details the criterion schema.

The judge also reports confidence. A leaf signal is used only at confidence 2 or 3; lower-confidence leaves are treated as missing, so they do not enter precision but can reduce recall when their omission leaves fewer than the required number of members. Because grading re-fetches cited pages, solving and grading should occur close together: a URL that changes after submission can cause an otherwise correct record to fail.

Identity resolution Before aggregation, the grader resolves identity independently along each key axis. Closed or predictable axes are canonicalized, while open-ended axes are semantically deduplicated; this prevents cosmetic variants from satisfying a volume floor more than once and makes coverage well-defined across parent tasks and subtasks. In the running example, the company key is resolved across the appointment and listing branches, while the composite company–appointee key distinguishes appointments. Appendix B details the canonicalization, deduplication, and entity-scope policies.

Metrics Leaf scores aggregate *bottom-up*. At each level, precision averages every supplied child score. Recall first collapses duplicate identities by retaining the worst score for each entity, then sorts the distinct entity scores, keeps the top k , where k is the required count, and zero-pads any shortfall before averaging. The grader reports soft and hard versions of precision, recall, and F1:⁵

⁵More detail is provided by the stylized rollup-algorithm implementation in Appendix F and by a rollup-walkthrough example rendered via the repository-shipped viewer in Appendix G.

- **Precision** — the mean score among submitted members. Soft precision gives partial credit to incomplete members; hard precision assigns credit only when a submitted member’s required subtree is fully correct.
- **Recall** — the sum of retained member scores divided by the required member count, measuring quality-adjusted completion relative to the target. If a submission exceeds the target, only the top k members by score are retained; if it falls short, the remainder is zero-padded. Soft recall gives partial credit, while hard recall assigns credit only to submitted members whose required subtrees are fully correct.
- **F1** — the harmonic mean of precision and recall within each task. Soft F1 summarizes partial-credit collection quality; hard F1 summarizes complete-member performance.

The benchmark reports the simple unweighted mean of each per-task metric. Every task therefore contributes equally to the headline score, regardless of its required record count.

These metrics localize failure along several dimensions. *Precision* measures the quality of submitted members, so low precision is consistent with a depth failure such as incorrect facts, unfaithful excerpts, or missing corroboration. The gap between scores under `verdict_retrieval` and `verdict_full` measures how much additional credit is lost when grading the complete submitted record rather than page-level task satisfaction alone. The drop from precision to recall is primarily a breadth signal: recall zero-pads unmet quotas and collapses duplicate identities, whereas precision averages the submitted children. A system that submits only a few perfect members can therefore earn high precision but low recall. Poor identity management can inflate raw submitted-member counts, but canonicalization and deduplication collapse aliases before recall is computed. The soft–hard gap is a completeness signal: hard scores remove partial credit from submitted members whose required subtrees are incomplete.

6 Experiments and Results

6.1 Setup

Conceptually, evaluation uses a four-stage record-level pipeline—*solve* → *fetch* → *judge* → *score* (Figure 6). *Solve* converts every system’s output into a common record format, ensuring identical downstream grading. *Fetch* retrieves the cited pages and identifies broken or bot-walled results, including login walls, paywall stubs, and unrendered JavaScript. Triage routes these cases through a heavier JavaScript-rendering browser before judgment, reducing crawler-induced failures. In parallel, the evaluation pipeline canonicalizes and deduplicates each key axis. *Judge* issues one verdict per record. *Score* aggregates those verdicts into task-level metrics; during recall aggregation, duplicate identity variants collapse to one entity at their worst score. Append-only caching allows interrupted runs to resume rather than restart. Every released task packages the same pipeline.

We evaluate six production systems with different search and orchestration interfaces (Table 3). These external systems are evaluated only after the task set is frozen and are not used for admission or rejection. The comparison includes task/search application programming interfaces (APIs; Exa and Parallel), general web-agent APIs (OpenAI’s Responses API with web search and a code interpreter; Anthropic’s managed agents), a deep-research agent (Gemini Deep Research), and a programmatic search-orchestration system (Perplexity Search as Code). All systems use the same pinned GPT-5.4 evaluation configuration: low effort for triage and canonicalization, medium for judging, and high for deduplication. Composite tasks multiply matching main-task and subtask scores, so a weak subtask reduces the composed entity score. By contrast, fan-out over a dispatch axis is averaged: covering three of four evidence types earns 0.75, while a missing subtask cell can zero the corresponding entity.



Figure 6: The grading pipeline. **Solve** normalizes each system’s output into the common record format; **fetch** retrieves every cited page, with triage routing broken fetches to a browser retry; **judge** canonicalizes and deduplicates identities and emits a structured judgment for each record; **score** aggregates verdicts into soft and hard precision, recall, and F1 scores at the task level (Section 5). The evaluated solver interfaces are described under Setup (Table 3).

System	Solver surface	Configuration	Model / setting	Output delivery
Perplexity	Programmable primitives / orchestration (Search as Code)	Production Agent API	GPT-5.5 / high	File-sharing tool
Anthropic	General tool-agent endpoint	Managed Agents beta, 2026-04-01 toolset	Opus 4.8 / high	Sandbox file access
OpenAI	General tool-agent endpoint	Responses API, web search, code interpreter	GPT-5.5 / high	Sandbox file access
Gemini	Product deep-research interface	Gemini Deep Research, April 2026 preview	Deep Research / speed	Output tokens
Parallel	Hosted task/search endpoint	Parallel Tasks production API, processor <code>ultra4x</code>	— / <code>ultra4x</code>	Output tokens
Exa	Hosted task/search endpoint	Exa Agent production API, effort <code>high</code>	— / <code>high</code>	Output tokens

Table 3: Systems evaluated in the main benchmark run. Configurations, model settings, and output-delivery mechanisms are the recorded run settings; when the interface exposes no effort override, we report the selected model’s documented default. Output delivery records how the harness obtains the large JSON submission: through provider sandbox files, Perplexity’s file-sharing tool, or model output tokens. Solver-surface labels describe how the benchmark invokes each system; they are not product tiers or capability classes.

6.2 Main Results

We report two sets of main results. On the full sample, we use the second-highest available setting for Perplexity, OpenAI, Parallel, and Exa, Gemini’s `speed` setting,⁶ and Anthropic’s `high` setting,⁷ together with each system’s best available delivery method. On a 45-task subset, we evaluate every available effort setting. Appendix H reports the delivery-method ablation on the same subset.

Full sample We report one run per system over all 500 benchmark tasks. Scores are simple, unweighted means of per-task metrics. Trials ending in terminal errors after repeated retries have no metric-bearing verifier result and are zero-filled in the aggregate.⁸

System	Completed	Soft			Hard		
		Precision	Recall	F1	Precision	Recall	F1
Perplexity	500	0.389	0.357	0.363	0.150	0.134	0.133
Anthropic	500	<u>0.354</u>	<u>0.222</u>	<u>0.249</u>	<u>0.137</u>	<u>0.073</u>	<u>0.072</u>
OpenAI	<u>499</u>	0.149	0.115	0.121	0.049	0.036	0.035
Gemini	498	0.132	0.042	0.055	0.062	0.007	0.009
Parallel	496	0.154	0.055	0.069	0.054	0.016	0.019
Exa	500	0.130	0.057	0.070	0.042	0.018	0.020

Table 4: WANDR scores under `verdict_full` for the main benchmark run. **Completed** is the scheduled 500 tasks minus errored trials and therefore equals the number of metric-bearing trials. Soft and hard precision, recall, and F1 are copied directly from the recorded job-level metrics; no score is reconstructed from another metric. Bold marks the best value in each numeric column and underline marks the second-best distinct value.

⁶We used Gemini `speed` instead of `max` on the full sample to save time and cost; `max` is evaluated in a 45-task subset.

⁷Claude Managed Agents exposes no effort override in its agent configuration [Anthropic, 2026], so we report effort from the selected model. Claude Opus 4.8 defaults the `effort` parameter to `high` across all surfaces, including the Messages API. See Anthropic’s May 28, 2026 release notes: <https://platform.claude.com/docs/en/release-notes/overview#may-28-2026>.

⁸Retries target specific failure modes: provider/API retries handle request failures, Relay permits up to two full solver restarts, and verifier retries address incomplete judgments. A trial still errors if every attempt ends in a terminal provider failure, timeout, or missing required output.

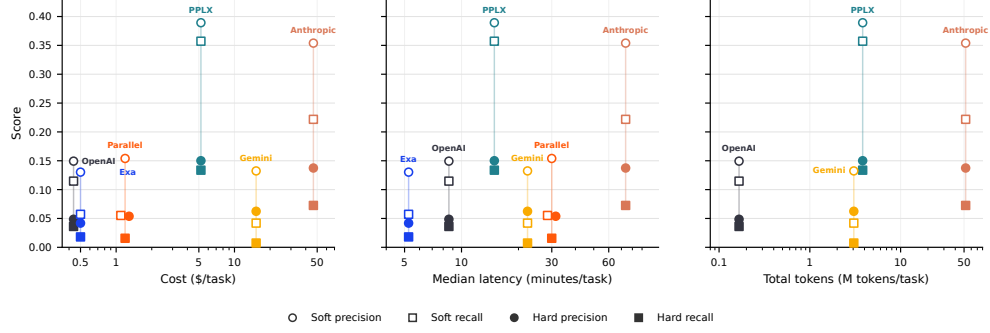
System	Completed	Cost \$/task	Latency		Token usage	
			Med. min	P90 min	In M	Out k
Perplexity	500	5.20	14.9	37.1	3.79	29.6
Anthropic	500	46.43	73.7	147.6	51.75	316.3
OpenAI	499	0.50	8.6	<u>12.9</u>	0.13	<u>33.7</u>
Gemini	498	15.24	22.3	62.9	<u>2.96</u>	91.8
Parallel	496	1.19	30.0	71.1	–	–
Exa	500	<u>0.50</u>	5.3	9.9	–	–

Table 5: Operational statistics for the same benchmark runs as Table 4. **Completed** is the scheduled 500 tasks minus errored trials. \$/task is average solver cost. Med. min and P90 min are the median and 90th-percentile solve-stage latencies computed from first-to-last solve-stage timestamps, excluding verifier time. In M and Out k are average provider-reported input and output tokens per scheduled task, in millions and thousands; Exa and Parallel do not expose token counters. Bold marks the best value and underline marks the second-best distinct value, using unrounded values; higher is better for completion and lower is better for resource use.

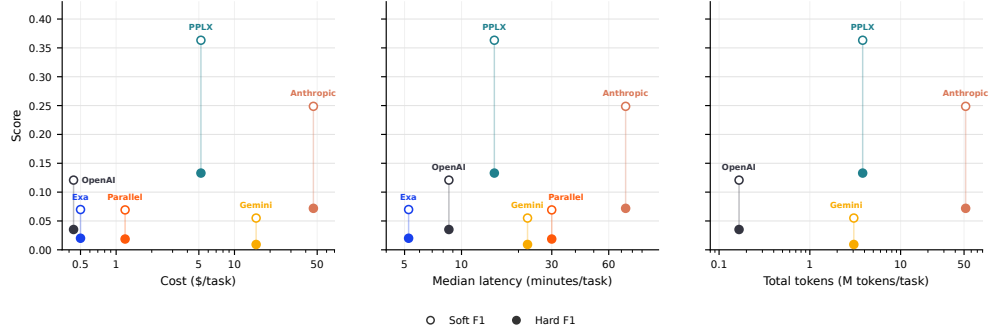
Taken together, Tables 4 and 5 and Figure 7 show that no system dominates both performance and resource use. Perplexity leads at 0.363 soft F1 and 0.133 hard F1 with midrange cost (\$5.20 per task, 14.9-minute median latency, and 3.82M total reported tokens per task). Anthropic ranks second (0.249, 0.072) but has the highest cost, latency, and token use. OpenAI and Exa cost about \$0.50 per task, and Exa is fastest at a 5.3-minute median, but the remaining systems reach at most 0.121 soft F1 and 0.035 hard F1. Exa and Parallel do not expose token counts. The benchmark remains unsaturated: the highest mean task-level hard precision and recall are only 0.150 and 0.134, corresponding to complete credit for roughly one in seven submitted members and one in seven required members, respectively, under the benchmark’s task-level averaging.

45-task subset We evaluate every system–setting pair under the systems’ best delivery channels, as in Table 3. Tables 6 and 7 report the results. Perplexity, OpenAI, Parallel, and Exa each have four settings; Gemini has two and Anthropic one `high` configuration. *Completed* is the number of metric-bearing trials out of 45.

Taken together, Tables 6 and 7 and Figure 8 extend the comparison across all available effort settings. Perplexity, Gemini, and Exa improve monotonically in soft and hard F1 across their available settings; Perplexity `xhigh` reaches 0.447 soft F1 and 0.224 hard F1. OpenAI peaks at `high`, while Parallel’s soft F1 rises through `ultra8x` but its hard F1 dips slightly between `ultra2x` and `ultra4x`. Resource use spans more than four orders of magnitude in cost, from Exa `low` at \$0.03 per task to Gemini `max` at \$324.83.



(a) Soft and hard precision and recall.



(b) Soft and hard F1.

Figure 7: Performance-cost views under `verdict_full` for the main benchmark run. Each column uses a conventional log x-axis, so movement up and left is better; system colors are consistent throughout the paper. In (a), circles and squares denote precision and recall, while white and solid fill denote soft and hard metrics; in (b), fill denotes soft and hard F1. Small horizontal offsets are visual only; tables and prose report the unshifted values. Scores are the task-level means in Table 4. Token use is the reported total per task; Exa and Parallel are omitted because they expose no token counters.

System	Setting	Completed	Soft			Hard		
			Precision	Recall	F1	Precision	Recall	F1
Perplexity	low	45	0.214	0.109	0.121	0.098	0.051	0.053
Perplexity	medium	45	0.350	0.289	0.295	<u>0.166</u>	0.156	0.149
Perplexity	high	45	<u>0.414</u>	<u>0.391</u>	<u>0.397</u>	0.162	<u>0.160</u>	<u>0.156</u>
Perplexity	xhigh	<u>44</u>	0.459	0.449	0.447	0.242	0.249	0.224
Anthropic	high	45	0.365	0.238	0.262	0.155	0.095	0.099
OpenAI	low	45	0.106	0.032	0.038	0.027	0.024	0.024
OpenAI	medium	45	0.158	0.087	0.091	0.067	0.055	0.053
OpenAI	high	45	0.177	0.150	0.153	0.076	0.081	0.073
OpenAI	xhigh	45	0.148	0.128	0.127	0.063	0.069	0.060
Gemini	speed	45	0.127	0.036	0.048	0.068	0.015	0.020
Gemini	max	45	0.106	0.069	0.074	0.049	0.025	0.028
Parallel	ultra	40	0.013	0.004	0.005	0.005	0.002	0.002
Parallel	ultra2x	45	0.156	0.050	0.063	0.069	0.021	0.026
Parallel	ultra4x	45	0.179	0.053	0.067	0.077	0.020	0.025
Parallel	ultra8x	45	0.170	0.068	0.080	0.075	0.032	0.035
Exa	low	45	0.136	0.004	0.006	0.128	0.002	0.003
Exa	medium	45	0.054	0.009	0.013	0.025	0.003	0.004
Exa	high	45	0.151	0.058	0.073	0.058	0.026	0.029
Exa	xhigh	45	0.142	0.103	0.111	0.039	0.035	0.036

Table 6: Matched 45-task score sweep across all available system settings. Every score is a simple zero-filled mean over the same 45 scheduled tasks. Bold marks the best value in each numeric column and underline marks the second-best distinct value, using unrounded values.

System	Setting	Completed	Cost \$/task	Latency		Token usage	
				Med. min	P90 min	In M	Out k
Perplexity	low	45	0.40	<u>1.2</u>	3.4	<u>0.10</u>	3.3
Perplexity	medium	45	2.07	5.8	19.3	1.01	13.7
Perplexity	high	45	4.75	15.6	36.9	4.11	28.2
Perplexity	xhigh	<u>44</u>	7.32	18.3	32.3	7.69	43.2
Anthropic	high	45	49.09	78.5	132.7	57.04	316.6
OpenAI	low	45	<u>0.07</u>	1.2	2.1	0.02	4.3
OpenAI	medium	45	0.32	5.4	8.0	0.10	19.5
OpenAI	high	45	0.49	8.8	13.8	0.14	32.1
OpenAI	xhigh	45	0.74	10.4	14.4	0.16	55.3
Gemini	speed	45	14.03	20.6	58.3	2.60	83.1
Gemini	max	45	324.83	92.9	322.7	73.62	896.1
Parallel	ultra	40	0.27	26.6	55.3	–	–
Parallel	ultra2x	45	0.60	23.7	37.8	–	–
Parallel	ultra4x	45	1.20	34.9	66.2	–	–
Parallel	ultra8x	45	2.40	31.3	58.1	–	–
Exa	low	45	0.03	1.2	5.9	–	–
Exa	medium	45	0.10	6.3	11.5	–	–
Exa	high	45	0.50	5.3	7.2	–	–
Exa	xhigh	45	1.00	9.1	17.1	–	–

Table 7: Operational statistics for the same matched 45-task sweep as Table 6. Cost and token totals are divided by 45 scheduled tasks; median and P90 are solve-stage latencies over tasks with valid timestamps. In M and Out k denote millions and thousands of tokens. Exa and Parallel expose no token counters. Bold marks the best value and underline marks the second-best distinct value, using unrounded values; higher is better for completion and lower is better for resource use.

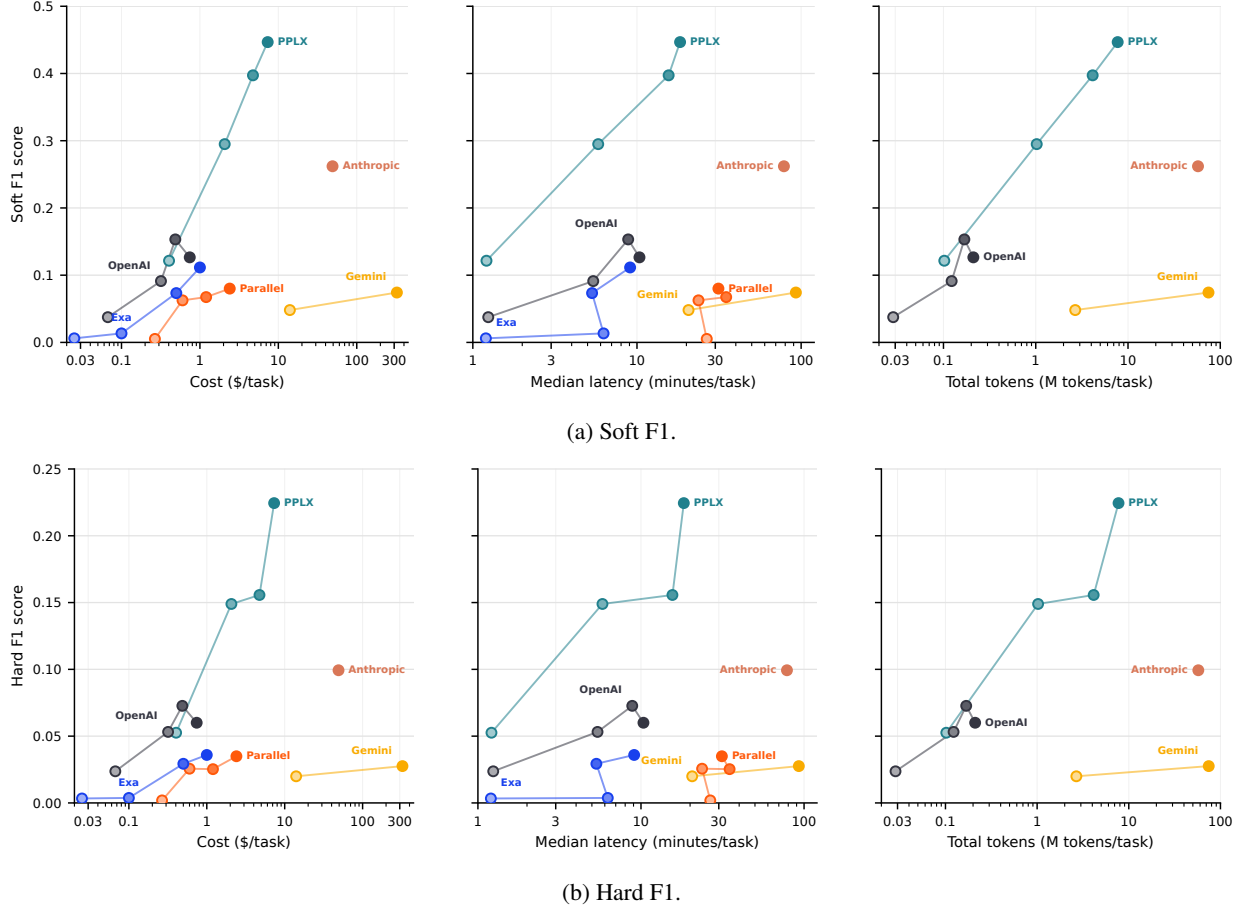


Figure 8: F1 against cost, median solve latency, and total token use for the matched 45-task effort sweep in Tables 6 and 7. Every point in (a) and (b) uses the same 45 tasks; for settings also run on the full benchmark, only the matched 45 tasks are included. System names directly label the colored paths, and darker color indicates higher effort. Exa and Parallel are absent from the token columns because their results expose no token counters.

6.3 Failure Analysis

6.3.1 WANDR as a Five-Layer Retrieval Program

A WANDR task naturally forms a retrieval program: partition the required entity space, fan out source-specific queries, fetch candidates in parallel, enforce source and type constraints, join evidence back to entities, deduplicate, track target attainment, and backfill sparse branches. Search as Code (SaC) exposes retrieval, ranking, filtering, fan-out, and rendering as software development kit (SDK) primitives that a model composes through generated Python in a sandbox [Perplexity, 2026]. The sandbox handles deterministic operations such as batching, retries, joins, aggregation, and deduplication without requiring a separate model turn for every web operation. Filesystem serialization can also carry candidate and attainment tables across turns without repeatedly placing all intermediate state in model context.

We decompose the program into five layers. *Pipeline synthesis* chooses source classes, query partitions, filters, joins, and stopping rules. *Breadth-and-depth execution* runs that plan at the required scale, completes every downstream branch, and backfills sparse ones. *Identity disambiguation* canonicalizes keys, collapses duplicates, and joins evidence to the correct entity across branches. *Semantic qualification* decides whether a candidate has the requested type, eligibility, source role, and page-level substance. *Evidence rendering* extracts a faithful, sufficient excerpt and emits the normalized record. SaC gives the model direct control over pipeline synthesis and deterministic compute for repeated execution; explicit keys and structured extraction primitives can also support identity, qualification, and rendering. It does not, however, make the generated plan or its semantic decisions automatically correct: an over-narrow

source template can miss an entire branch, an over-broad filter can admit invalid candidates, a faulty identity join can attach the wrong evidence, and an optimistic stopping rule can leave the required set incomplete. Code can therefore make correct control flow repeatable, but it can also apply a bad decision consistently at scale.

SaC matches WANDR’s repeated horizontal retrieval structure more directly than a serial search-and-read loop. Perplexity’s profile is consistent with this: it combines the smallest soft precision–recall gap (Table 4) with the lowest excerpt-faithfulness and excerpt-requirement failure rates (Figure 11), without incurring the highest observed cost, latency, or token use. This is observational support rather than a controlled ablation because the systems also differ in models, skills, endpoints, and search infrastructure.

This five-layer decomposition is conceptual, and the benchmark localizes the layers at different resolutions. Structural counts directly measure breadth-and-depth completion; row-level checks separate semantic qualification from evidence rendering. Duplicate clusters let us separate the count effect of identity collapse from raw under-delivery, while Appendix G shows how take-worst scoring and cross-branch composition propagate an identity error. The current observables still do not distinguish a faulty synthesized plan from incomplete execution of a sound plan across systems, so pipeline-synthesis errors—including programmatic error amplification—remain hypotheses rather than a separate empirical category.

6.3.2 Breadth-and-Depth Execution

Precision–recall gaps expose missing breadth Under `verdict_full`, soft recall is lower than soft precision for every system (Table 4). Precision averages the quality of submitted members; recall evaluates those scores against the required member count, collapsing duplicate identities and zero-padding any shortfall. Perplexity has the smallest gap, retaining 0.357 recall from 0.389 precision. OpenAI’s gap is almost as small (0.149 to 0.115), but both values are much lower, so a narrow gap alone does not imply strong coverage. Anthropic shows the largest separation, from 0.354 precision to 0.222 recall. Duplicate collapse contributes to this difference, but its all-task count effect is only 0.017–0.205 percentage points across systems (Table 8); under-delivery is therefore the dominant aggregate interpretation. The verifier trees localize the missing volume before final URL attachment (Figure 9).⁹ Mean raw discovery completion ranges from 0.611 to 0.951, below conditional enrichment (0.767–0.967) and terminal evidence-slot completion (0.979–0.994). Perplexity leads raw discovery at 0.951 and enrichment at 0.967, concentrating its structural advantage before evidence attachment.

Soft–hard gaps expose incomplete members Soft metrics reward partial subtrees, whereas hard metrics credit only members whose required descendants and checks are all complete. The conversion is severe for every system: Perplexity falls from 0.363 soft F1 to 0.133 hard F1, Anthropic from 0.249 to 0.072, and the remaining systems fall from 0.055–0.121 to 0.009–0.035 (Table 4). The loss is not only a recall effect: Perplexity’s precision drops from 0.389 soft to 0.150 hard, and Anthropic’s from 0.354 to 0.137, showing that many submitted members remain only partially complete. Programmable fan-out and backfilling address breadth, but hard completion also requires coordinated success in identity resolution, semantic qualification, and evidence rendering. SaC still leads on every hard metric, but it does not eliminate the gap.

Complete performance collapses as targets grow From the smallest to largest required-volume bin, Perplexity’s hard precision falls from 0.235 to 0.096 and hard recall from 0.219 to 0.079; Anthropic falls from 0.205 to 0.113 and from 0.127 to 0.042; and OpenAI falls from 0.109 to 0.034 and from 0.102 to 0.016 (Figure 10, top). Gemini is the precision exception, rising from 0.063 to 0.088, but its hard recall still approaches zero (0.023 to 0.001). Bigger targets therefore reduce both the quality of fully complete submitted members and, more sharply, completion relative to the requested set. Resource use does not rise uniformly enough to explain the pattern: Anthropic’s median latency and mean cost increase substantially; the same quantities for Perplexity stay nearly flat until the largest bin and remain approximately flat for OpenAI, Parallel, and Exa.

Complete performance collapses as hierarchies deepen From no intermediate key to three or more, Perplexity’s hard precision falls from 0.392 to 0.019 and hard recall from 0.378 to 0.017; Anthropic falls from 0.311 to 0.049 and from 0.229 to 0.022; and OpenAI falls from 0.136 to 0.012 and from 0.120 to 0.011 (Figure 10, bottom). The

⁹We use top-level keys as a proxy for discovery and intermediate keys as a proxy for enrichment. This is usually faithful to the task shape but is approximate: members can be composite, and a top-level key can instead be a partition.

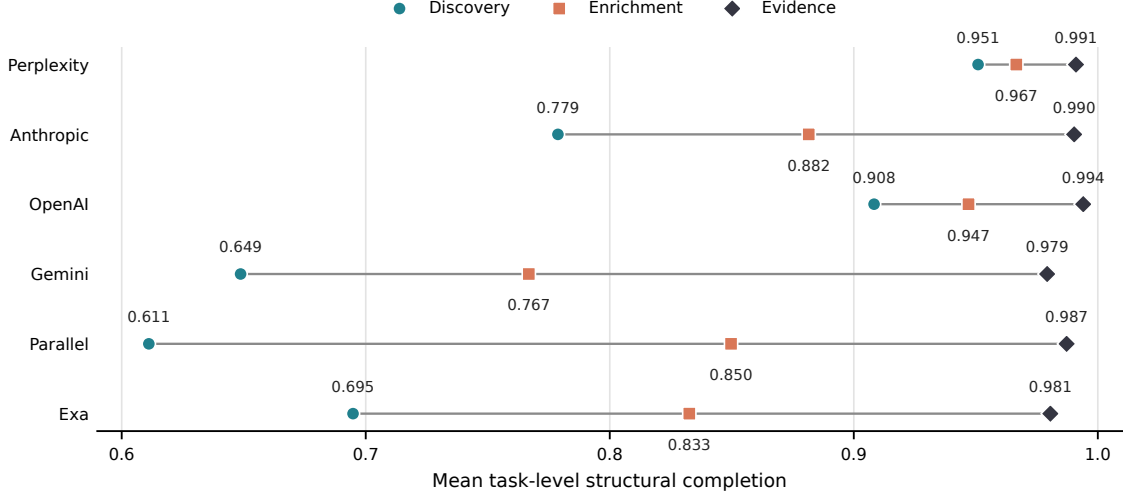


Figure 9: Mean task-level raw structural completion from the verifier trees. Circles show discovery, squares enrichment, and diamonds evidence; lines connect hierarchy levels within a system and do not represent temporal trajectories. For each key-level node, completion is $\min(\text{provided}, \text{required}) / \text{required}$, where *provided* is the pre-collapse count; nodes are averaged within each trial and then trials are averaged with equal weight. Discovery is the top key, enrichment covers intermediate keys, and evidence is the terminal key. Enrichment averages only tasks with an intermediate level. Underlying verifier details are available for discovery/enrichment/evidence on 500/437/496 Perplexity tasks, 500/441/500 Anthropic, 499/439/498 OpenAI, 493/435/493 Gemini, 350/312/350 Parallel, and 490/433/490 Exa. Table 8 reports the distinct-count adjustment. Values are shown to three decimals.

other systems finish at or below 0.028 hard precision and 0.001 hard recall. Additional levels therefore damage complete-member quality even among submitted members, not only target-relative completion: every extra branch creates another opportunity for a member to fail the hard conjunction. Cost and latency show no common monotonic response. These curves are observational stratifications rather than controlled scaling experiments because hierarchy depth co-varies with target volume, task content, and difficulty.

6.3.3 Identity Disambiguation

Duplicate collapse is sparse globally but severe locally The score tree records both the raw number of provided values and the merged duplicate clusters at every key level. For a non-URL key-level node with raw provided count p , duplicate excess d (all variants beyond one representative per cluster), and required count r , we isolate the count component of identity loss as

$$\ell_{\text{id}} = \frac{\min(p, r) - \min(p - d, r)}{r}. \quad (3)$$

Table 8 reports an equal-task mean of this quantity across entity-key nodes, plus the conditional mean over nodes with $d > 0$. This count decomposition is conservative with respect to the full score: it does not include the additional take-worst quality penalty within a merged cluster.

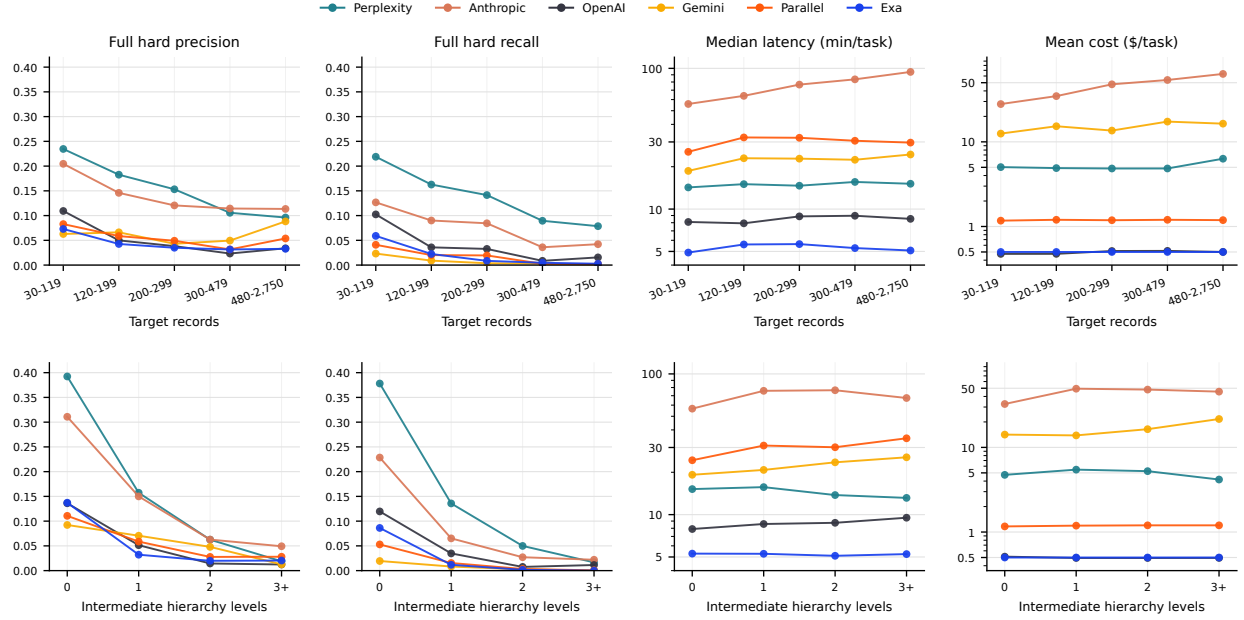


Figure 10: Task-level hard precision, hard recall, and resource scaling across target record volume (top) and intermediate hierarchy levels (bottom). Each line is one fixed main-run system configuration; points are equal-task aggregates within bins. Hard precision and recall under `verdict_full`, together with mean solver cost, use the scheduled bin denominator, with missing metric or cost values zero-filled to match the benchmark aggregate; latency is the median over tasks with valid solve timestamps. The target-volume bins contain 87, 103, 86, 117, and 107 tasks, and the intermediate-level groups contain 69, 237, 161, and 33 tasks; 3+ pools three to five intermediate keys between the member and URL levels. Cost and latency use log y-axes. Lines connect bins for readability and do not imply a causal effect of either structural variable.

System	Tasks	Duplicate collapse		Count loss (pp)	
		Affected (%)	Values (%)	All tasks	Affected nodes
Perplexity	500	4.8	0.426	0.205	20.8
Anthropic	500	2.4	0.018	0.017	9.0
OpenAI	499	7.0	0.353	0.125	22.3
Gemini	493	5.5	0.086	0.030	10.3
Parallel	350	8.0	0.184	0.057	13.4
Exa	490	6.3	0.191	0.048	11.1

Table 8: Identity collapse on non-URL key axes in the six full runs. Tasks gives the available verifier-tree coverage. Affected is the share of tasks with at least one merged entity-key cluster; Values is excess variants divided by all pre-collapse provided entity-key values. The All tasks column averages Equation 3 within each task and then equally across tasks; the Affected nodes column averages only nodes containing a merged cluster and then equally across affected tasks. Values are derived from the shipped S3 verifier reports.

Across systems, only 2.4%–8.0% of tasks contain an entity-key duplicate cluster, and just 0.018%–0.426% of pooled provided entity-key values collapse. The all-task count loss is correspondingly small (0.017–0.205 percentage points), so duplicate collapse is not the dominant aggregate structural deficit. Locally, however, an affected key node loses 9.0–22.3 percentage points of distinct-count completion on average. Repeated URL values are rarer still: they affect 0.8%–1.6% of tasks and collapse 0.009%–0.059% of provided URL values. Appendix G shows the complementary score effect: take-worst retains the weakest duplicate variant, and canonical identity determines which evidence branches compose before the penalty propagates upward.

SaC concentrates rather than eliminates identity errors Perplexity has entity-key clusters on 4.8% of tasks, below Gemini, Exa, OpenAI, and Parallel, but it has the highest pooled collapsed-value share (0.426%) and the second-largest affected-node loss (20.8 percentage points). OpenAI has the largest affected-node loss (22.3 percentage points), while Parallel has the highest affected-task share (8.0%). SaC is therefore not uniformly advantaged at the identity layer: its failures are less frequent than in four systems but more concentrated when they occur. This pattern is consistent with generated code repeatedly applying a mistaken key or join rule within a task, but the final score trees do not establish that mechanism causally.

6.3.4 Semantic Qualification

Usable, valid pages are common; complete page support is not For five systems, only 3.2%–8.9% of submitted pages are unusable; OpenAI is the 23.1% outlier (Figure 11). Invalid-or-wrong-type failures are also comparatively limited at 4.2%–17.4%. The larger problem begins after a page is fetched and judged plausible: 33.6%–68.3% of records cite a page that does not satisfy every substantive task requirement. Anthropic performs best on this check at 33.6% failure, with Perplexity second at 41.4%. Finding an accessible, broadly in-scope page is therefore not the main semantic bottleneck; finding a page that establishes the complete requested claim is.

Incomplete excerpt support is the largest component failure Excerpt-requirement failure ranges from 57.5% to 86.6%, the highest component failure rate for every system and within 0.6 percentage points of final-verdict failure in every case (Figure 11). Faithfulness alone does not explain the deficit. Perplexity has the lowest excerpt-faithfulness failure rate at 19.3%, yet its excerpts still fail to support every requirement on 57.5% of records; Anthropic shows the same separation, 26.5% versus 59.2%. Perplexity has the lowest measured failure on both excerpt checks, but even its submitted excerpts are incomplete on more than half of judged records. The dominant downstream problem is therefore not merely copying text accurately; it is selecting enough evidence to substantiate the full claim. These pooled row-level rates describe each system’s submitted-record mix and are not task-matched causal estimates.



Figure 11: Measured row-level failure probes from available verifier details, in percent (lower is better). Page req. unmet means that the fetched page fails at least one substantive requirement; Excerpt req. unmet means that the submitted excerpts fail at least one. Final verdict denotes failure under `verdict_full`. These diagnostics are not headline task scores. Each cell reports the value rounded to one decimal place, and row labels give the number of available task details. Parallel has details for 350 tasks, so its field analysis is less complete than its aggregate results.

6.3.5 Evidence Rendering

The downstream gap persists at task level A related conversion gap appears in task scores when we compare `verdict_retrieval` in Equation 2 with `verdict_full` in Equation 1. The retrieval-only verdict asks whether

the fetched page satisfies every substantive requirement, independently of the submitted excerpts. The full verdict additionally requires the universal checks, task-specific validity, and complete excerpt support. We report the absolute loss, $\Delta = F1_{\text{retrieval}} - F1_{\text{full}}$, and its retrieval-relative percentage, $\Delta\% = 100\Delta/F1_{\text{retrieval}}$. Because both verdicts grade the same submissions, their operational statistics are identical.

System	Soft F1				Hard F1			
	Retrieval-only	Full	Δ	$\Delta\%$	Retrieval-only	Full	Δ	$\Delta\%$
Perplexity	0.531	0.363	0.168	31.6%	0.245	0.133	0.112	45.6%
Anthropic	<u>0.436</u>	<u>0.249</u>	0.187	43.0%	<u>0.181</u>	<u>0.072</u>	0.109	60.2%
OpenAI	0.280	0.121	0.159	56.8%	0.094	0.035	0.059	62.6%
Gemini	0.139	0.055	0.084	60.4%	0.033	0.009	0.024	72.0%
Parallel	0.170	0.069	0.100	59.2%	0.071	0.019	0.052	73.5%
Exa	0.260	0.070	0.190	73.1%	0.098	0.020	0.078	79.5%

Table 9: F1 under the retrieval-only and full verdicts over all 500 tasks. Δ is retrieval-only minus full F1, and $\Delta\%$ is that loss divided by retrieval-only F1. F1 values are copied directly from the recorded job-level metrics; the two diagnostic losses are calculated from those values. Bold and underline mark the best and second-best retrieval-only and full values; losses are not ranked.

Every system scores lower under `verdict_full` than under `verdict_retrieval` (Table 9). The absolute soft-F1 loss ranges from 0.084 to 0.190, and the hard-F1 loss from 0.024 to 0.112; relative losses range from 31.6% to 73.1% for soft F1 and 45.6% to 79.5% for hard F1. Perplexity remains highest under both verdicts and has the smallest relative loss, but its 0.168 soft-F1 and 0.112 hard-F1 losses show that page-level task satisfaction does not reliably become a complete submitted record. Exa has the largest relative conversion loss under both soft and hard F1. The comparison is consistent with the excerpt-support pattern above, but it does not isolate excerpts causally because `verdict_full` adds several checks at once.

6.3.6 Cross-System Synthesis

The shared pattern is a sequence of compounding losses. Recall falls below precision, hard scores fall far below soft scores, performance deteriorates as target volume and hierarchy depth increase, substantive page support fails much more often than page usability or basic validity, and incomplete excerpt support is the largest record-level component failure (Table 4, Figures 10 and 11). Perplexity combines the smallest soft precision–recall gap with the lowest excerpt-faithfulness and excerpt-requirement failure rates, but it is only second-best on page requirements and its identity failures are severe when they occur. Anthropic has the lowest page-requirement failure and identity-collapse prevalence, yet shows a much larger precision–recall gap and substantial excerpt incompleteness. OpenAI is the page-usability outlier and has the largest affected-node identity loss; Gemini shows pronounced precision–recall collapse and high page-requirement failure; Parallel has the highest identity-affected task share and frequent excerpt failures, although its diagnostic coverage is limited to 350 tasks; and Exa has the largest relative loss from `verdict_retrieval` to `verdict_full` (Table 8 and Table 9). SaC’s distinctive signature is therefore stronger breadth retention and excerpt construction, not uniformly better performance at every layer.

7 Discussion

7.1 Summary

WANDR evaluates a form of research common in professional work but poorly represented by single-answer and report-writing benchmarks: constructing a large structured collection while independently verifying every member. Its 500 tasks combine a large-scale, often open-ended breadth requirement with a per-record depth requirement, and its qualification key hierarchy makes discovery, enrichment, and evidence failures separately observable. Reference-free grading accommodates changing facts without maintaining an exhaustive gold answer table; each submitted record instead carries the page and excerpt needed for verification.

The evaluation shows that this problem remains far from solved. Across six production systems, Perplexity Search as Code leads at 0.363 soft F1 and 0.133 hard F1, but even its hard recall is only 0.134. Its profile is consistent with

what programmable fan-out, batching, filtering, joining, deduplication, and target-aware backfilling would predict, although the cross-system comparison cannot identify a causal component: Perplexity has the smallest soft precision–recall gap and the lowest excerpt-faithfulness and excerpt-requirement failure rates. Its depth performance is not uniformly best, however. Anthropic has the lowest page-requirement failure rate, and even Perplexity’s cited pages and excerpts fail complete support on 41.4% and 57.5% of judged records, respectively. The common bottleneck is therefore not search alone but complete evidence construction at scale.

The five-layer framework directly measures four layers at aggregate or task-local resolution. Breadth-and-depth execution produces both precision–recall and soft–hard gaps, and complete performance drops sharply with larger targets and deeper hierarchies. Identity disambiguation is sparse in aggregate but can sharply reduce distinct completion on affected nodes. In semantic qualification, page usability and basic validity are usually not the main failures; substantive page support is. Evidence rendering then creates the largest component loss because submitted excerpts frequently fail to support every requirement. Pipeline synthesis is not causally isolated. More generally, the results support treating wide research as a systems problem coupling retrieval, deterministic data processing, identity management, semantic qualification, state management, and evidence rendering.

Even though WANDR is primarily an evaluation benchmark, the same structure makes it a plausible RL substrate. The task pipeline can generate held-out training packages, per-level required counts support curriculum learning, record-level verdicts and hierarchy-level scores provide denser outcome rewards than a single exact match, and the streaming grader can amortize overlapping work across batched rollouts. Appendix I discusses this design and its caveats, including the need to train on generated or separately held-out tasks rather than the released benchmark split.

7.2 Limitations

Scope and representativeness The 500 released tasks are designed to represent recurring patterns in de-identified production requests, but they are not a random sample of all research work. Admission deliberately favors high-volume, feasible, discriminative tasks with independently verifiable web evidence. This makes WANDR useful as a stress test, but the resulting score distribution should not be interpreted as the difficulty of research requests in deployment.

System comparison and causal attribution The main evaluation reports one production run per system rather than repeated trials to save time and cost. An earlier, separately configured evaluation reported the same qualitative ordering [Perplexity, 2026], but it also used one run per system and therefore does not estimate run-to-run variance. Systems differ simultaneously in model, search infrastructure, agent harness, tool interface, and provider-side implementation. The cross-system ranking is therefore observational and does not isolate any individual component.

Reference-free grading Evidence verification avoids a manual and static gold table but introduces judge and retrieval uncertainty. The grader uses LLM judge calls and re-fetches each cited page: the page may drift after solving, and extraction can omit dynamic, tabular, or JavaScript-rendered content. Canonicalization and deduplication introduce additional boundary decisions when identities are ambiguous. We keep solving and grading close in time, and retry broken pages with a browser, but these controls reduce rather than eliminate grading error.

Agentic fetch backend alignment The grader first re-fetches cited pages through an agentic fetch backend that is also available to the Perplexity solver; unsuccessful fetches are routed to a browser. Because this backend is shared, the grading path may be better aligned with pages that the Perplexity solver can access than with pages reached through other systems’ retrieval stacks. In internal grading ablations that disable the agentic fetch backend and use only the browser, scores decrease for every system while the ranking remains mostly stable. The grading fetch path therefore affects absolute scores and may confer some alignment advantage, even though the comparative ordering appears less sensitive. Future evaluations should report the grading fetch configuration and compare multiple retrieval paths.

7.3 Future Research Directions

Controlled scaling studies Figure 10 identifies descriptive scale sensitivity in the current fixed-configuration runs, but target volume, intermediate-level count, content, and difficulty co-vary. A useful next experiment is a matched comparison that varies one factor at a time: workload scale, model, search interface, sandbox, skills, or retrieval infrastructure.

Mechanism-resolved failure analysis The failure analysis is observational: score trees and verifier fields identify where submitted outputs lose credit, but they do not establish which model, retrieval, orchestration, or interface mechanism caused the loss. Controlled ablations should hold the task sample and the rest of the system fixed while varying one component at a time, such as code execution, parallel fan-out, target-aware backfilling, context persistence, identity handling, or the fetcher. Paired runs with repeated trials would distinguish causal effects from task composition and run-to-run variation.

RL training experiments Appendix I explains how WANDR can serve as a training environment; the next step is to test the claim. Experiments can compare terminal F1 alone against denser, record- and branch-level rewards, measure whether quota curricula improve performance, and quantify the grading savings from shared queues and caches as task-by-rollout batch size grows. Training should use generated sibling tasks or newly generated tasks from appropriate held-out seeds, preserve a held-out benchmark split, and test for reward hacking—especially over-submission, evidence templating, and exploitation of judge or cache artifacts.

7.4 Conclusion

WANDR makes large-scale, evidence-backed research measurable without an exhaustive gold collection. Across 500 tasks, recall falls below precision, hard scores remain far below soft scores, and complete performance degrades sharply as target volume and hierarchy depth increase. Page usability and basic validity usually succeed, but substantive page support and especially complete excerpt support remain major bottlenecks. The released tasks, verifier, and structural diagnostics provide a basis for improving comprehensive search systems and testing whether those gains hold at scale.

References

- Anthropic. Claude managed agents overview. <https://platform.claude.com/docs/en/managed-agents/overview>, 2026. Accessed: 2026-07-13.
- Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. Finance agent benchmark: Benchmarking LLMs on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*, 2025.
- Shan Chen, Pedro Moreira, Yuxin Xiao, Sam Schmidgall, Jeremy Warner, Hugo Aerts, Thomas Hartvigsen, Jack Gallifant, and Danielle S Bitterman. Medbrowsecomp: Benchmarking medical deep research and computer use. *arXiv preprint arXiv:2505.14963*, 2025.
- Boyue Gou, Zanning Huang, Yuting Ning, Yu Gu, Michael Lin, et al. Mind2Web 2: Evaluating agentic search with agent-as-a-judge. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2025.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, et al. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents. *arXiv preprint arXiv:2601.20975*, 2026.
- Ziyang Huang, Haolin Ren, Xiaowei Yuan, Jiawei Wang, Zhongtao Jiang, Kun Xu, Shizhu He, Jun Zhao, and Kang Liu. WideSeek: Advancing wide research via multi-agent scaling. *arXiv preprint arXiv:2602.02636*, 2026.
- Satyapriya Krishna, Kalpesh Krishna, Anhad Mohanane, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqi. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4745–4759, 2025. doi: 10.18653/v1/2025.naacl-long.243. URL <https://aclanthology.org/2025.naacl-long.243/>.
- Tian Lan, Bin Zhu, Qianghuai Jia, Junyang Ren, Haijun Li, Longyue Wang, Zhao Xu, Weihua Luo, and Kaifu Zhang. DeepWideSearch: Benchmarking depth and width in agentic information seeking. *arXiv preprint arXiv:2510.20168*, 2025.
- Tian Lan, Felix Henry, Bin Zhu, Qianghuai Jia, Junyang Ren, Qihang Pu, Haijun Li, Longyue Wang, Zhao Xu, and Weihua Luo. Table-as-Search: Formulate long-horizon agentic information seeking as table completion. *arXiv preprint arXiv:2602.06724*, 2026.
- Haitao Li, Junjie Chen, Jingli Yang, Qingyao Ai, Wei Jia, Youfeng Liu, Kai Lin, Yueyue Wu, Guozhi Yuan, Yiran Hu, et al. LegalAgentBench: Evaluating LLM agents in legal domain. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2322–2344, 2025. doi: 10.18653/v1/2025.acl-long.116. URL <https://aclanthology.org/2025.acl-long.116/>.
- Shunyu Liu, Minghao Liu, Huichi Zhou, Zhenyu Cui, Yang Zhou, Yuhao Zhou, Jialiang Gao, Heng Zhou, Yunhao Yang, et al. VeriWeb: Verifiable long-chain web benchmark for agentic information-seeking. *arXiv preprint arXiv:2508.04026*, 2025.
- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations*, 2024.
- Perplexity. Rethinking search as code generation, 2026. URL <https://research.perplexity.ai/articles/rethinking-search-as-code-generation>. Perplexity Research. Accessed: 2026-06-19.
- Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- Ryan Wong, Jiawei Wang, Junjie Zhao, et al. WideSearch: Benchmarking agentic broad info-seeking. *arXiv preprint arXiv:2508.07999*, 2025. ByteDance Seed.

- Lei Xiong, Kun Luo, Ziyi Xia, Wenbo Zhang, Jin-Ge Yao, Zheng Liu, Jingying Shao, Jianlyu Chen, Hongjin Qian, et al. AutoResearchBench: Benchmarking AI agents on complex scientific literature discovery. *arXiv preprint arXiv:2604.25256*, 2026.
- Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. Assistant-Bench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968, 2024. doi: 10.18653/v1/2024.emnlp-main.505. URL <https://aclanthology.org/2024.emnlp-main.505/>.
- Joey Zhong, Hao Zhang, Clare Southern, Jeremy Yang, Thomas Wang, Kate Jung, Shu Zhang, Denis Yarats, Johnny Ho, and Jerry Ma. DRACO: A cross-domain benchmark for deep research accuracy, completeness, and objectivity. *arXiv preprint arXiv:2602.11685*, 2026. URL <https://arxiv.org/abs/2602.11685>.
- Junting Zhou, Wang Li, Yiyan Liao, Nengyuan Zhang, Tingjia Miao, Zhihui Qi, Yuhao Wu, and Tong Yang. ScholarSearch: Benchmarking scholar searching ability of LLMs. *arXiv preprint arXiv:2506.13784*, 2025.
- Fengbin Zhu, Xiang Yao Ng, Ziyang Liu, Chang Liu, Xianwei Zeng, Chao Wang, Tianhui Tan, Xuan Yao, Pengyang Shao, Min Xu, et al. FinDeepResearch: Evaluating deep research agents in rigorous financial analysis. *arXiv preprint arXiv:2510.13936*, 2025.
- Yutao Zhu, Xingshuo Zhang, Maosen Zhang, Jiajie Jin, Liancheng Zhang, Xiaoshuai Song, Kangzhi Zhao, Wencong Zeng, Ruiming Tang, Han Li, Ji-Rong Wen, and Zhicheng Dou. GISA: A benchmark for general information-seeking assistant. *arXiv preprint arXiv:2602.08543*, 2026.
- Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. Agent-as-a-Judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.

Appendices

A Per-Record Grading Criteria

Criteria define what one submitted record must prove. The judge evaluates each leaf independently from the submitted item, answer, URL, and excerpts, together with the fetched page.

A.1 Layered Checks

Each record is evaluated through universal checks, task-specific validity checks, and substantive requirements. Universal checks apply to every task. Validity checks decide whether the record is eligible and well-formed enough to judge. Requirements state the facts that the cited evidence must establish.

Every substantive requirement is evaluated twice. A field ending in `_satisfied` asks whether the full page supports the requirement, regardless of which excerpts the solver selected. Its paired `_supported` field asks whether the submitted excerpts alone are sufficient for a careful reader to verify the same requirement. The schema aggregates these pairs into `requirements_all_satisfied` and `requirements_all_supported`.

In the running CEO/CFO example, the appointment page must identify the company and appointee, establish the role, and place the first public announcement in the target window. A page can satisfy those requirements even when the solver submits only an excerpt naming the appointee. In that case, the page-level fields pass, but the excerpt-level date or company fields fail because the submitted evidence is incomplete.

Three universal checks guard this requirement pair:

- `page_content_usable` requires a reachable, substantive, on-topic page; dead links, stubs, and off-topic pages fail here.
- `answer_intent_clear` requires the record to state the specific claim being made. A page dump with no localized answer, or excerpts that contradict one another, fails here.
- `excerpts_faithful` requires every excerpt to appear verbatim or near-verbatim on the page with its meaning preserved. Paraphrase, fabricated text, misleading truncation, and sentence stitching fail this check.

Faithfulness and completeness are separate. An excerpt may quote the page exactly but omit the sentence that establishes the date, role, or source authority; it then passes `excerpts_faithful` but fails the relevant `_supported` field. Conversely, text stitched from separate passages may mention every needed fact but fail faithfulness because the page never states the combined claim. This separation is especially important when a solver relies on search snippets, which can omit qualifiers or join nearby text out of context.

Task-specific validity checks cover eligibility conditions that do not fit a page/excerpt requirement pair. Some are structural, such as a required excerpt length or allowed domain; others are broader sanity checks, such as whether a submitted person or company belongs to the requested class. The schema summarizes them in `overall_valid`. If a record is so invalid that substantive judgment would be meaningless, the judge short-circuits the remaining fields to false; otherwise, it still reports the requirement-level signals for diagnosis.

Figure 12 summarizes the resulting sequence. As defined in Section 5, `verdict_retrieval` asks whether the fetched page satisfies every substantive task requirement, independently of the submitted excerpts. Universal and validity fields remain available as diagnostics but are not multiplied into this verdict. The complete-record verdict, `verdict_full`, additionally requires validity, a usable page, a clear answer, faithful excerpts, and complete excerpt support. These are the two record-level verdicts aggregated by the task metrics.

A.2 Common Evidence Patterns

The exact requirements vary by task, but several evidence patterns recur across the benchmark:

- *Plausible entity* — the submitted entity must be a sensible instance of the requested class. A quote author should be a real person rather than a fictional character; an LLM producer should build models rather than merely resell access to them.
- *Verified eligibility* — some restrictions are key-level validity checks, while others require cited evidence. In the running example, the appointment branch checks that the company is US-based, while the listing subtask requires listing-authority evidence of public status.



Figure 12: The record-level checks. `verdict_retrieval` reads the full-page requirement signal, while `verdict_full` also requires a usable page, a clear answer, faithful excerpts, complete excerpt support, and task-specific validity. A task-specific validity check can short-circuit an ineligible record.

- *Appropriate source* — the citation must come from a source suited to the claim, such as an official filing, first-party product page, recognized listing authority, or practitioner forum. The allowed source type can be established by domain, page authorship, or in-page evidence.
- *Long-tail source* — the qualifying evidence may sit outside the highest-ranked results. These tasks reward systematic search beyond prominent pages and aggregators.
- *Heavy source* — the evidence may live in a long report, registry, filing, or PDF. Verifying the claim requires fetching the artifact and navigating to the relevant passage rather than relying on a snippet.
- *Dedicated source* — the page must be substantially about the submitted entity, not a directory row, search-result page, or passing mention. For example, a task may require a dedicated product listing rather than a buyer’s guide that names the product once.
- *Broad extraction* — the claim itself may require many pieces of evidence. Showing that a governing body has at least N members, for example, requires excerpts that enumerate enough members even without an explicit word-count floor.
- *Substantive extraction* — the page and excerpts must contain analysis or detail, not merely a rating, label, or short listing. A narrative-review requirement filters out bare aggregate scores.
- *Scattered extraction* — the required evidence appears in different parts of a page, such as a title, date, and substantive comment. The solver must collect all of it without joining unrelated passages.
- *Structured extraction* — exact values sit in tables, filings, or specification sheets. The main risk is reading the wrong cell, row, or column; GPU–game FPS results are a typical example.

B Identity Resolution

Identity rules determine when two submitted values count as the same member. Without them, cosmetic variants could satisfy a volume floor more than once: “Aleksandr Pushkin” and “Alexander Pushkin” should not count as two authors, nor should “NVIDIA” and “Nvidia Corp.” count as two companies.

Leaf verdicts remain independent, but scoring must compare values across the submission to count distinct members. WANDR limits this cross-record step to one sameness policy per key axis. Once identity is resolved on each axis, required volumes, coverage, and score rollups are well defined.

B.1 Canonicalization and Deduplication

WANDR uses two complementary mechanisms:

- *Canonicalization* maps a submitted value to a standard form. It is well suited to closed or predictable sets: “Mar. 2025” and “March 2025” can both map to “2025-03,” dispatch labels can map to a fixed vocabulary, and URLs can be normalized mechanically. A closed-set canonicalizer can also reject values outside the allowed set.
- *Deduplication* decides whether two values refer to the same entity. Exact matching is sufficient after deterministic normalization for fields such as URLs or fixed labels. Open-ended entities such as people and companies usually require semantic comparison.

Both mechanisms are configured per key axis. Each key must contain enough information to identify its values without relying on an implicit parent. Composite keys supply that context when one field is ambiguous. In the running example, `company_appointee` includes both the company and appointee fields, distinguishing appointments of the same person—or same-named people—across different companies. A single-field key is sufficient when its values are already unambiguous within the task.

When a subtask reuses a parent key, both branches share one identity axis. In the running example, company names from the appointment and listing branches are resolved together before the two branch scores are composed.

B.2 Common Identity Patterns

Identity policy is part of the task definition, not merely cleanup. Changing what counts as the same entity changes the set the task asks the solver to cover.

- *Granularity* — the policy sets the level being counted. Folding franchise locations and subsidiaries into one identity creates a brand-level task; keeping them separate creates an outlet- or company-level task. A closed set may deliberately mix levels when the task treats them as peers, such as a roster containing states, the District of Columbia, and selected county-level regimes.
- *One referent across forms* — renames, abbreviations, and different stages of the same event can map to one underlying member. A product can remain the same offering after a rebrand, and coverage of a transaction’s announcement and closing can refer to the same transaction even though the pages use different language.
- *Exact-set coverage* — a finite canonical set changes “find at least n ” into “cover every listed value.” Aliases map to the same allowed member, and values outside the solver-facing roster do not satisfy the axis.

C Task Package Format

Each released WANDR task is a self-contained package. Complete package files, prompts, fixtures, and verifier code live in the public repository.

Component	What it contains	Why it matters
Solver-facing task text	The natural-language request shown to a system under test.	Defines the agent’s obligation without exposing the judge checklist as a separate answer key.
Key hierarchy	The tree of identifying keys ending in <code>url</code> , with required child counts at each level.	Determines the member unit, breadth, depth, zero-padding, and where a failure is localized.
Volume floors	Per-level <code>required</code> counts and, where relevant, closed sets or open “all that qualify” settings.	Enforces coverage without enumerating a complete gold answer table.
Judge schema	Universal fields plus task-specific validity gates and requirement pairs.	Specifies what the verifier checks on the cited page and in the submitted excerpts.
Identity policy	Canonicalization for closed-form keys and deduplication policy for open-ended identities.	Prevents duplicate padding and resolves surface variants of the same member.
Calibration fixtures	Small records with intended verdicts, including positive and negative cases.	Unit-tests the task’s judge before the task can enter the benchmark set.
Metadata and labels	Structural labels, distribution labels, and release bookkeeping.	Supports dataset statistics, curation, and reproducible benchmark filtering.
Verifier harness	Shared fetch, judge, identity-resolution, and scoring code pinned with the release.	Lets outside users regrade submissions under the same task-level mechanics, subject to live-web and judge-model dependencies.

Table 10: WANDR task package format.

D Task Admission Gates and Review

The authoring prompts and linter instructions are implementation artifacts in the release repository. A task must be runnable, convention-clean, sufficiently difficult, discriminative, empirically feasible, and judgeable before it can enter the released set.

Gate	Check	Failure action
Runs	The package loads and executes under the public harness with its fixtures.	Return to task or fixture authoring
Clean	The solver-facing description, judge specification, and schema state the same requirements in the same order; mechanical lints pass.	Patch mechanical issues or return to authoring
Difficulty	Primitive internal baselines do not already achieve high completion.	Harden the task or reject an easy seed
Discriminative	Weak internal baselines score low while stronger internal rollouts score meaningfully higher.	Tighten task or reject shortcut-prone seed
Feasibility audit	A merged solution over 10–12 authoring rollouts recovers a near-full requested volume floor.	Adjust volume or rewrite scope
Judge audit	A reviewer blindly regrades sampled records and attributes disagreements to solver behavior, task design, or grader machinery.	Fix judge or block task
Human rubric	Where automatic checks do not settle the decision, reviewers assess the key hierarchy, task text, judge, and identity handling.	Return to owning stage

Table 11: Admission and review gates for a task package. Curation follows admission and selects from the admitted pool to match release-level distributions.

Review axis	Substantive question
Key hierarchy	Are keys sensible, are volume floors achievable and nontrivial, and is URL corroboration neither missing nor excessive?
Task authorship	Does the solver-facing prose unambiguously entail the graded obligation without hidden requirements or shortcuttable ambiguity?
Judge authorship	Does the judge schema encode the right validity gates and page/excerpt requirement checks?
Identity handling	Are canonicalization and deduplication configured for the kind of identity being counted?

Table 12: Human-review rubric.

Reviewers also inspect the rollouts and trajectories accumulated during authoring. This exposes degenerate solution strategies and shortcuts that may not be apparent from the package or aggregate scores alone, and lets reviewers return the task for hardening before admission.

E Exemplar Task Coverage

The authoring pipeline supplies a broader set of vetted guidance tasks to the author, critic, and linter. The five tasks profiled here are a deliberately varied subset that makes their design rationale explicit. They provide orientation, not templates: during design, the author can map a seed to a nearby exemplar archetype but may not copy its structure. Table 13 summarizes the distinct benchmark shapes covered by this profiled subset, followed by task cards describing the requested records and claims.

Exemplar	Key hierarchy sketch	Member unit	Records	Depth / identity pattern	Stress tested
gpu_benchmarks	GPU → game → URL; subtasks for GPU price and game rating	GPU–game benchmark cell	290	Matrix fill with subtasks and corroborated enrichment; GPU/game deduplication and URL canonicalization	First-hand benchmark tables, gapless coverage, no second-hand roundup shortcut
bachir_doha_brand_competitors	Brand → facet → URL	Brand–facet finding	400	Open-set discovery plus four-part dispatch; brand deduplication and facet canonicalization	Source-role discipline: review pages, menus, social pages, and positioning pages are not interchangeable
adult_guardianship_programs	Closed jurisdiction root plus legislation-signal subtask	Jurisdictional program or legal signal	780	Closed member panel with dispatch-like legal comparison areas; jurisdiction canonicalization	Legal authority, statutory fit, and multi-area enrichment over a fixed jurisdiction set
devtools_partnerships	Company → partner → reference type → URL	Ordered partnership reference	600	Asymmetric pair identity with quote/backquote dispatch	Reciprocity, official-source control, and duplicate corporate-family traps
llm_presence	Company → month → site → URL(2)	Company–month–site discussion cell	720	Temporal panel with site canonicalization and two-page corroboration per cell	Date anchoring, source diversity, and first-hand discussion filtering

Table 13: Coverage summary for the five exemplar tasks. The sketches intentionally abstract away the full package files, which are released in the repository.

gpu_benchmarks

Task: For at least 20 GPUs and 10 games per GPU, find a source-backed FPS result (one URL per GPU–game pair) from a first-hand benchmark or review page. Two enrichment subtasks require each GPU’s current retail price and each game’s public review rating, with three URLs per item.

Record: an FPS figure for one GPU–game pair, backed by a benchmark page.

Records required: 290 (200 cells + 60 price + 30 review).

Claims per record:

1. The benchmark is for the claimed GPU model — *proof: the benchmark page*
2. The benchmark is for the claimed game title, a real released game — *same page*
3. The page confirms a concrete FPS score — *same page*
4. Per GPU, its current retail price; per game, its review rating — *proof: three retailer or review pages per item (subtasks with corroboration)*

bachir_doha_brand_competitors

Task: Identify 100+ dessert brands operating in Doha or elsewhere in Qatar (ice cream, gelato, frozen dessert, sweets, bakery dessert, or dessert café) and cover four analysis facets per brand—owned social identity, customer sentiment, delivery commerce, and market positioning—with one focused source per facet.

Record: one facet finding for one brand, backed by a facet-appropriate page.

Records required: 400 (100 brands × 4 facets).

Claims per record:

1. The brand is a real, locally operating, in-scope Qatar brand — *validity*
2. The page ties the brand to the Qatar market — *proof: the facet page*
3. The page genuinely plays the claimed facet’s source role (a review page cannot serve *market positioning*) — *dispatch-routed check*
4. The page exposes a focused finding for that facet — *same page*

adult_guardianship_programs

Task: For all 30 listed jurisdictions, including the District of Columbia, identify the public or last-resort adult-guardianship arrangement (one URL each). A legislation subtask then finds five legal signals in each of five comparison areas per jurisdiction: appointment basis, decision-support alternatives, selection priority, oversight, and rights review.

Record (root): a jurisdiction's institutional fallback arrangement, backed by a statute, agency, or program page.

Records required: 780 (30 root + $30 \times 5 \times 5$ subtask).

Claims per record:

1. The named arrangement is the jurisdiction's institutional fallback for adults needing a guardian of last resort — *proof: the cited page*
2. The jurisdiction is one of the 30 items in the closed list — *closed-set validity*
3. (subtask) each legal signal's cited section fits its comparison area, on an authoritative legal surface — *proof: own page per signal, dispatch-specific*

devtools_partnerships

Task: For at least 100 developer-tools or DevOps companies, find three advertised partners whose relationship is acknowledged in both directions. For every ordered (*company*, *other_company*) pair, provide both a *quote* (the company naming the partner) and a *backquote* (the partner meaningfully acknowledging the company), with one URL for each direction.

Record: one direction of one partnership—a page on the referencing party's own surface naming the counterpart.

Records required: 600 ($100 \times 3 \times 2$).

Claims per record:

1. The page is on a surface officially controlled by the referencing party — *proof: the page (incl. its host)*
2. It names the opposite party of the pair — *same page*
3. The reference is substantive for its type—a backquote must show meaningful reciprocity, not a logo wall or name-drop — *per-type check*
4. The counterpart is a genuine, distinct company (not an alias or the same corporate family) — *validity*

llm_presence

Task: Identify 10+ LLM-producer companies and build a community-sentiment panel: for each company and each month in a fixed 12-month window, provide dedicated first-hand discussion or impression pages from three different sites, with two URLs per (company, month, site) cell.

Record: one standalone community-discussion page about one company in one month on one site.

Records required: 720 ($10 \times 12 \times 3 \times 2$).

Claims per record:

1. The page is clearly dated to the target month (in-content dates, post timestamps, or thread markers) — *proof: the page*
2. It is on the claimed site — *same page (incl. host)*
3. It is a standalone, first-hand impression or discussion of the company—not a landing page, bulk thread view, or search-results display — *same page*
4. Two independent such pages per cell — *corroboration ($k=2$)*

F Stylized Metric Rollup Algorithm

Metric computation recursively follows the task structure (Section 3). *Criteria* supply the leaf score selected from the two verdicts defined in Section 5. *Topology* specifies which levels exist, what volume each requires, and where subtasks compose. *Identity* determines which submitted values represent the same entity within each level. The rollup is parameterized by three choices: aggregation (*precision* or *recall*), continuity (*soft* or *hard*), and verdict. A subtask score is multiplied into the matching parent entity's score using the same choices and recursive procedure; a missing subtask value contributes zero.

```
def score(task, level, identifiers, params):
    records = task.records.narrow(identifiers)
    if not records:
        return 0

    if level + 1 == len(task.keys):
        return records[0].judgment[params.verdict] # leaf judgment
    next_key = task.keys[level + 1].name

    values = {r[next_key]: r["cluster_id"] for r in records}
    c_scores = [
        score(task, level + 1, identifiers | {next_key: v}, params)
        for v in values
    ]

    if params.aggregation == "recall":
        volume = task.keys[level + 1].volume
        clusters = duplicates(values, of=next_key)
        c_scores = pad(top(take_worst(c_scores, clusters), volume), 0, volume)
        s = mean(c_scores)

    if level == -1: # thresholding and composition don't apply at overall-task avg
        return s

    if params.continuity == "hard":
        s = floor(s)

    curr_key = task.keys[level].name
    sub_scores = [
        score(sub, 0, {curr_key: identifiers[curr_key]}, params)
        for sub in task.subtasks(level)
    ]

    return s * product(sub_scores)

metric = score(root_task, -1, {}, params)
```

Stylized metric rollup. The released task packages include the production implementation.

G Worked Rollup Walkthrough

We illustrate the rollup with a toy composite task scored end to end by the released grader and rendered in the submission viewer included with each task package. The task requests three countries, two cities with more than one million inhabitants per country, and one supporting page per city—country (3) $\rightarrow$ city (2) $\rightarrow$ url (1). It adds three subtasks: a per-city `heritage_site` (2) $\rightarrow$ url (1) branch requesting two UNESCO-linked heritage sites, plus two per-country URL-only branches requesting evidence of current or historical female leadership and of hosting a worldwide sporting event. Score indicators show hard and soft precision and recall under `verdict_full`. The submission covers the three countries at deliberately varied levels of completeness and obtains soft F1 0.57 and hard F1 0.33 (Figure 13). Figures 14–16 expand the three country subtrees.

The Turkey subtree isolates the two identity effects summarized in Table 8: duplicate collapse reduces the distinct count, and take-worst can lower the retained cluster score before canonical identities match main-task and subtask evidence branches.

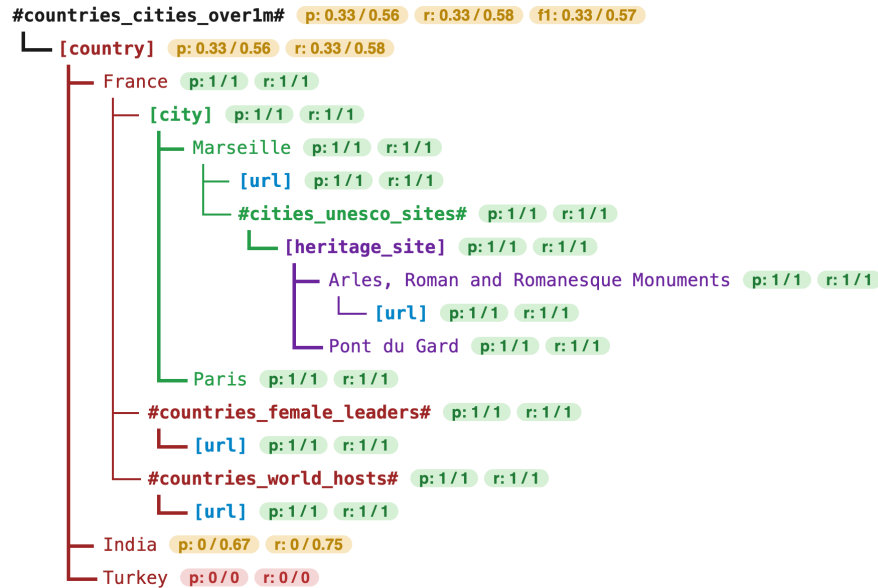


Figure 13: Overall score for the toy composite task. France is complete, India receives partial credit, and Turkey fails a required subtask, yielding soft F1 0.57 and hard F1 0.33.



Figure 14: France is fully complete: both qualifying cities have two heritage sites, and both country-level subtasks pass. All displayed precision and recall values equal one.



Figure 15: India illustrates partial recall and surplus handling. Delhi supplies one of two required heritage sites, so zero-padding leaves soft recall at 0.5 while soft precision remains 1. Three cities are supplied for a required count of two; Bangalore's empty heritage subtask gives it a zero composed score, so it is excluded when recall retains the two highest-scoring cities.



Figure 16: Turkey illustrates identity collapse and subtask gating. At the heritage level, take-worst retains an ambiguous zero-scoring citation when it merges the two Selimiye Mosque variants, reducing Istanbul’s recall. The city-level aliases Istanbul and Istanbul, Türkiye then merge, propagating the lower score. The worldwide-sporting-event subtask also fails because its evidence concerns a continental final, so multiplicative composition sets the country score to zero.

H Output-Delivery Ablation

The output ablation compares solutions delivered through sandbox files, a file-sharing tool, or output tokens. All six rows use the same 45 tasks and the same score and operational columns as the full sweep. These are independent roll-outs rather than paired deterministic executions, so the differences are descriptive associations with both the delivery configuration and the particular rollout.

System	Delivery	Completed	Soft			Hard		
			Precision	Recall	F1	Precision	Recall	F1
Perplexity	share	45	0.414	0.391	0.397	<u>0.162</u>	0.160	0.156
Perplexity	output	45	0.331	0.224	0.238	0.169	<u>0.116</u>	<u>0.121</u>
Anthropic	sandbox	45	<u>0.365</u>	<u>0.238</u>	<u>0.262</u>	0.155	0.095	0.099
Anthropic	output	42	0.305	0.176	0.202	0.116	0.073	0.080
OpenAI	sandbox	45	0.177	0.150	0.153	0.076	0.081	0.073
OpenAI	output	45	0.189	0.109	0.122	0.087	0.068	0.068

Table 14: Score comparison for the matched 45-task output-delivery ablation at each system’s selected configuration. Bold marks the best value in each numeric column and underline marks the second-best distinct value.

System	Delivery	Completed	Cost \$/task	Latency		Token usage	
				Med. min	P90 min	In M	Out k
Perplexity	share	45	4.75	15.6	36.9	4.11	28.2
Perplexity	output	45	5.08	15.4	33.4	3.79	48.5
Anthropic	sandbox	45	49.09	78.5	132.7	57.04	316.6
Anthropic	output	<u>42</u>	38.05	64.7	227.3	39.54	305.1
OpenAI	sandbox	45	0.49	<u>8.8</u>	<u>13.8</u>	0.14	<u>32.1</u>
OpenAI	output	45	<u>0.68</u>	8.8	11.9	<u>0.15</u>	49.5

Table 15: Operational statistics for the same matched output-delivery ablation as Table 14. Cost and token totals are divided by 45 scheduled tasks; median and P90 are solve-stage latencies over tasks with valid timestamps. In M and Out k denote millions and thousands of tokens. Bold marks the best value and underline marks the second-best distinct value, using unrounded values; higher is better for completion and lower is better for resource use.

Output-token runs have lower soft F1 than file-delivery runs for Perplexity (0.397 versus 0.238), Anthropic (0.262 versus 0.202), and OpenAI (0.153 versus 0.122); hard F1 shows the same ordering (0.156 versus 0.121, 0.099 versus 0.080, and 0.073 versus 0.068). The absolute soft-F1 loss decreases monotonically with the file-delivery score across these three systems (0.159, 0.060, and 0.031). This three-system association is descriptive rather than causal, but it suggests that output-only systems may be somewhat disadvantaged while also making it unlikely that delivery alone explains the much larger gaps in the main ranking. Operational differences vary by system: output-token runs are slightly more expensive for Perplexity and OpenAI and less expensive per scheduled task for Anthropic, whose output-token run also produces fewer metric-bearing trials (42/45 versus 45/45).

I Suitability for Reinforcement Learning

WANDR is released primarily as an evaluation benchmark. Its structure nevertheless offers several properties relevant to future training splits and harnesses.

Scalable task supply and diversity The semi-automated task pipeline avoids exhaustive answer annotation through a reusable author–critic–linter process and reference-free verification. It can produce new tasks and controlled siblings across domains, hierarchy shapes, evidence requirements, traps, and breadth/depth settings, while curation steers the resulting distribution (Section 4). This creates a path to training-scale data with broader variation than repeated sampling of the 500 released tasks. For RL-scale task supply, authoring and admission can run fully automatically, including feasibility and judge-quality gates; periodic human audits can monitor distribution drift without sitting on the generation path. To preserve evaluation validity, RL should use newly generated or separately held-out task packages, not the released benchmark split.

A natural curriculum Workload is explicit in each task’s key hierarchy: every level declares a required count. Holding the task’s semantic criteria fixed while lowering one or more counts produces a smaller instance—for example, requiring 5 qualifying entities rather than 20, or one corroborating source rather than three—and the counts can be increased across epochs toward the full task. The same mechanism can stage breadth and depth separately, allowing a curriculum to move from reliable single-record evidence construction to sustained collection building. Count reduction controls workload rather than guaranteeing monotonic semantic difficulty, and each generated variant should still pass the normal feasibility and judge checks.

Dense, decomposable reward A long rollout need not receive only a terminal all-or-nothing score. Each submitted record receives `verdict_retrieval` and `verdict_full`, and the score tree aggregates those verdicts by entity and hierarchy level into soft and hard precision, recall, and F1. Soft metrics reward correct partial progress when the requested collection is incomplete; hard metrics preserve pressure to complete whole members; and the gap between the two verdicts separates page-level task satisfaction from complete record construction. A trainer can therefore assign outcome rewards at record and branch granularity or use universal and task-specific sub-verdicts as auxiliary signals while retaining task-level F1 as the objective. This supervision is denser than a single exact-match reward, though moving from outcome-level to process supervision would require nontrivial trajectory backtracking and heuristics for assigning credit from individually graded final records to the actions responsible for them.

Amortized batch grading Reference-free verification avoids constructing and maintaining exhaustive gold collections. At runtime, expensive grading remains record-local until canonicalization, deduplication, and score rollup. The released grader accepts multiple task roots, flattens their records into one streaming queue graph, and executes fetch, triage, canonicalization, deduplication, and judgment with stage-specific concurrency. Fetches are batched, and queue nodes use both in-memory request coalescing and persistent caches keyed to the smallest reusable unit. Records citing the same URL can share one fetch result; identical triage, canonicalization, and judgment work can also be reused when task semantics and inputs match. This structure aligns with RL batches, where multiple tasks and rollouts may revisit the same pages and entities. Overlap among rollout records can therefore amortize grading work, while persistent caches limit resumed runs to missing work.

Training need	WANDR affordance	Caveat / required control
Held-out task supply	The task pipeline can generate siblings across domains, hierarchy shapes, source classes, traps, and breadth/depth settings without gold-answer enumeration.	Training should use generated or separately held-out packages, not the 500-task benchmark split.
Curriculum	Per-level <code>required</code> counts can be lowered or raised to stage breadth and depth separately.	Count reduction changes workload, not necessarily semantic difficulty; variants still need feasibility and judge checks.
Dense reward	<code>verdict_retrieval</code> , <code>verdict_full</code> , <code>page/excerpt</code> sub-verdicts, tree-level soft scores, and hard completion provide branch- and record-level outcome rewards.	Process supervision requires trajectory backtracking and action-level credit-assignment heuristics; reward shaping should be checked against task-level F1.
Batch grading	The grader’s record stream, staged concurrency, URL fetch sharing, and persistent caches can amortize overlapping records across tasks and rollouts.	The public entry point should add rollout/case identifiers and grouped score reporting for native task-by-rollout batches.
Reward-hacking audit	The verifier exposes over-submission, duplicate clusters, excerpt failures, page failures, and source-role failures.	Training experiments should explicitly test templated evidence, judge exploitation, cache artifacts, and benchmark-split leakage.

Table 16: RL-oriented view of WANDR’s task and grader design.